



باسمه تعالی

دانشگاه صنعتی شریف
دانشکده مهندسی برق

Introduction to Machine Learning Project

Hidden Markov Anomaly Detection

From OC-SVM to Structured Sequence Models

نیم سال دوم ۱۴۰۴-۱۴۰۵

دکتر سجاد امینی

هلیا تاج آبادی (403105063)

امیرعلی جهان بخشی (403105103)

فهرست مطالب

۲		<i>Data Sets of Project</i>	۱
۲		<i>Daphnet freezing of gait</i>	۱.۱
۶		<i>ECG</i>	۲.۱
۱۰		<i>Introduction to Anomaly Detection</i>	۲
۱۰		<i>Informal Definition</i>	۱.۲
۱۰		<i>Formal Problem Statement</i>	۲.۲
۱۱		<i>Density Level Set View</i>	۳.۲
۱۱		<i>Connection to This Project</i>	۴.۲
۱۱		<i>OC-SVM Baseline</i>	۳
۱۱		<i>Why OC-SVM?</i>	۱.۳
۱۲		<i>Model and Objective</i>	۲.۳
۱۳		<i>Interpretation of ν</i>	۳.۳
۱۵		<i>Implementation Requirements</i>	۴.۳
۲۰		<i>Latent Dependency and Real-World Motivation</i>	۴
۲۰		<i>Why the i.i.d. Assumption Can Fail</i>	۱.۴
۲۱		<i>Markov Chains</i>	۲.۴
۲۳		<i>Hidden Markov Models</i>	۳.۴
۲۴		<i>Most Likely State Sequence and Viterbi Decoding</i>	۴.۴
۲۶		<i>Connection to HMAD</i>	۵.۴
۲۷		<i>Bonus Questions</i>	۶.۴
۳۱		<i>Hidden Markov Anomaly Detection</i>	۵
۳۱		<i>Conceptual Bridge from OC-SVM to HMAD</i>	۱.۵
۳۷		<i>Hidden Markov Joint Feature Map</i>	۲.۵
۴۱		<i>Simplified HMAD Training Algorithm</i>	۳.۵
۴۵		<i>Prediction</i>	۴.۵
۴۹		<i>Conclusion</i>	۶

۱ Data Sets of Project

۱.۱ Daphnet freezing of gait

معرفی مجموعه داده

در این پروژه از مجموعه داده‌ی *Daphnet Freezing of Gait* استفاده شده است. این مجموعه داده با هدف بررسی و تشخیص خودکار پدیده‌ی توقف ناگهانی راه رفتن یا *Freezing of Gait (FoG)* در بیماران مبتلا به پارکینسون تهیه شده است.

پدیده‌ی *FoG* حالتی است که در آن بیمار برای یک بازه‌ی زمانی کوتاه، به صورت ناگهانی توانایی ادامه‌ی حرکت و راه رفتن را از دست می‌دهد. برای ثبت این پدیده، سه شتاب‌سنج سه محوره روی بخش‌های مختلف بدن افراد نصب شده‌اند:

- میچ پا؛
- ران؛
- ناحیه‌ی تنه.

در هر لحظه از زمان، هر شتاب‌سنج سه مؤلفه‌ی شتاب را در سه جهت مختلف ثبت می‌کند. بنابراین هر رکورد خام شامل زمان ثبت، نُه مقدار شتاب و یک برچسب وضعیت است. برچسب‌های اصلی مجموعه داده به صورت زیر تعریف شده‌اند:

- 0 = آزمایش اصلی بخش از خارج
- 1 = راه رفتن توقف رخداد بدون عادی حرکت
- 2 = راه رفتن ناگهانی توقف رخداد

منبع رسمی مجموعه داده

Roggen, D., Plotnik, M., & Hausdorff, J. (2010). *Daphnet Freezing of Gait*. UCI Machine Learning Repository.

DOI: 10.24432/C56K78

صفحه‌ی رسمی مجموعه داده:

UCI Machine Learning Repository – *Daphnet Freezing of Gait*

ماهیت دنباله‌ای داده‌ها و دلیل انتخاب آن

داده‌های این مجموعه داده دارای ساختار زمانی هستند؛ به این معنا که مشاهدات در لحظه‌های متوالی به یکدیگر وابسته‌اند. برای مثال، وقوع یک رخداد *FoG* معمولاً تنها در یک نمونه‌ی زمانی ظاهر نمی‌شود، بلکه برای چندین نمونه‌ی متوالی ادامه پیدا می‌کند. مثلاً چنین دنباله‌ای دیده می‌شود:

حرکت عادی ← شروع توقف حرکت ← ادامه‌ی توقف حرکت ← بازگشت به حرکت عادی.

بنابراین، ترتیب مشاهدات و نحوه‌ی انتقال میان وضعیت‌های مختلف اهمیت دارد. پس مجموعه داده‌ی *Daphnet* برای مقایسه‌ی مدل پایه‌ی *OC-SVM* با مدل آگاه از دنباله‌ی *HMAD* مناسب است.

جدول ۱: ساختار ویژگی‌های خام مجموعه داده‌ی *Daphnet*

شماره ستون	ویژگی	توضیح
0	زمان	زمان ثبت نمونه
1 تا 3	شتاب مج پا	سه مؤلفه‌ی شتاب در سه جهت مختلف
4 تا 6	شتاب ران	سه مؤلفه‌ی شتاب در سه جهت مختلف
7 تا 9	شتاب تنه	سه مؤلفه‌ی شتاب در سه جهت مختلف
10	برچسب	وضعیت عادی، رخداد <i>FoG</i> یا خارج از آزمایش

ویژگی‌های خام مجموعه داده

ویژگی استفاده شده در این پروژه

شتاب سنج‌های مورد استفاده در ثبت داده‌ها در آزمایش انجام شده شتاب سنج‌های سه محوره هستند و ما برای استفاده از داده‌های آن از شدت شتاب هر شتاب سنج استفاده کردیم یعنی اگر سه مؤلفه‌ی شتاب ران در لحظه‌ی t به ترتیب برابر با

$$a_{x,t}, \quad a_{y,t}, \quad a_{z,t}$$

باشند، ابتدا اندازه‌ی بردار شتاب محاسبه شده است:

$$m_t = \sqrt{a_{x,t}^2 + a_{y,t}^2 + a_{z,t}^2}.$$

سپس از آنجا که مقادیر اندازه‌گیری توسط شتاب سنج‌ها برحسب mg بودند، برای استاندارد سازی داده‌های بدست آمده را بر ۱۰۰۰ تقسیم کردیم و چون حتی در حالت سکون هم شتاب سنج حداقل شتاب گزارش زمین را اندازه میگیرد و یعنی حتی در سکون هم حداقل عدد ۱ را نشان میدهد، برای استاندارد سازی و اینکه در سکون ۰ را نشان دهد داده‌ها را یکی کم کردیم و در نهایت از این مقدارها استفاده میکنیم:

$$x_t = \frac{\sqrt{a_{x,t}^2 + a_{y,t}^2 + a_{z,t}^2}}{1000} - 1.$$

پس در نهایت از این داده‌های اسکالر استاندارد سازی شده استفاده میکنیم. (توجه کنید قبل از محاسبه شدت ابعاد هر مشاهده ما ۳ بود ولی حالا اسکالر شده)

تعریف دنباله و نمونه برداری

داده‌های اصلی هر بیمار شامل سری‌های زمانی بسیار طولانی هستند. برای تبدیل این داده‌ها به نمونه‌های مناسب برای خودمون، از روش پنجره‌بندی استفاده شده. هر دنباله در این پروژه به صورت زیر تعریف می‌شود:

$$\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_T^{(i)}],$$

که در آن طول هر دنباله برابر است با $T = 256$. بنابراین هر دنباله شامل 256 مشاهده‌ی زمانی متوالی از اندازه‌ی شتاب ران است:

$$\mathbf{x}^{(i)} \in \mathbb{R}^{256}.$$

نرخ نمونه برداری مجموعه داده برابر با 64 هرتز است؛ در نتیجه، هر دنباله تقریباً چهار ثانیه از حرکت فرد را نمایش می دهد. برای استخراج دنباله ها از یک پنجره ی لغزان با گام 128 استفاده شده است. بنابراین دو دنباله ی متوالی دارای 50 درصد هم پوشانی هستند تا ساختار زمانی را نیز خوب حفظ کنند.

روش تعیین دنباله های نرمال و ناهنجار

چون برای کوتاه کردن طول دنباله ها همان ابتدا از روش پنجره بندی استفاده کردیم، برای پیدا کردن لیبل هر دنباله این طور عمل میکنیم که برای هر دنباله، نسبت نمونه های زمانی دارای برچسب FoG محاسبه شده است:

$$r_{FoG} = \frac{\sum_{t=1}^T \mathbf{1}(l_t = 2)}{T},$$

که در آن l_t برچسب اصلی داده در لحظه ی t . برچسب نهایی هر دنباله براساس قانون زیر تعیین شده است:

$$y^{(i)} = \begin{cases} 0, & r_{FoG} = 0, \\ 1, & r_{FoG} \geq 0.25. \end{cases}$$

در این تعریف:

- $y^{(i)} = 0$ نشان دهنده ی دنباله ی نرمال است؛
- $y^{(i)} = 1$ نشان دهنده ی دنباله ی ناهنجار است.

روش تولید مجموعه های آموزش و آزمون

برای جلوگیری از نشت اطلاعات میان مجموعه های آموزش و آزمون، تقسیم داده ها براساس شناسه ی افراد انجام شده است، نه به صورت تصادفی روی دنباله ها. تقسیم افراد به صورت زیر بوده است:

$$\mathcal{S}_{\text{train}} = \{S01, S02, S03, S04, S05, S06\},$$

$$\mathcal{S}_{\text{test}} = \{S07, S08, S09, S10\}.$$

در مجموعه ی آموزش، فقط دنباله های کاملاً نرمال افراد $S01$ تا $S06$ استفاده شده اند. بنابراین هیچ دنباله ی ناهنجاری وارد فرایند آموزش مدل های تک کلاسه نشده است. مجموعه ی آزمون از دنباله های نرمال و ناهنجار افراد $S07$ تا $S10$ تشکیل شده است. نسبت ناهنجاری در مجموعه ی آزمون برابر با مقدار زیر در نظر گرفته شده است:

$$r_{\text{test}} = 0.1.$$

تعداد دنباله ها و توزیع برچسب ها

پس از اجرای مراحل آماده سازی و نمونه گیری، تعداد نهایی دنباله های آموزش و آزمون به صورت جدول زیر است: بنابراین ماتریس های داده دارای ابعاد زیر هستند:

$$X_{\text{train}} \in \mathbb{R}^{600 \times 256},$$

جدول ۲: تعداد دنباله‌های نرمال و ناهنجار

مجموعه	ناهنجار	نرمال	مجموعه
600	0	600	آموزش
600	60	540	آزمون
1200	60	1140	کل

$$X_{\text{test}} \in \mathbb{R}^{600 \times 256}.$$

تمام دنباله‌های مجموعه‌ی آموزش نرمال هستند، درحالی‌که مجموعه‌ی آزمون شامل 90 درصد دنباله‌ی نرمال و 10 درصد دنباله‌ی ناهنجار است. مجموعه‌داده شامل دنباله‌های با طول متغیر نیست و تمام ورودی‌ها ابعاد یکسانی دارند

مراحل پیش‌پردازش

مراحل پیش‌پردازش انجام‌شده روی مجموعه‌داده به‌ترتیب زیر هستند:

۱. خواندن مستقیم فایل‌های متنی موجود در فایل فشرده‌ی مجموعه‌داده؛
 ۲. شناسایی فرد مربوط به هر فایل براساس نام فایل؛
 ۳. تقسیم افراد به دو گروه مستقل آموزش و آزمون؛
 ۴. انتخاب سه محور شتاب‌سنج نصب‌شده روی ران؛
 ۵. محاسبه‌ی اندازه‌ی بردار شتاب سه‌محوره؛
 ۶. تبدیل واحد مقادیر شتاب از mg به مقیاس تقریبی g ؛
 ۷. کم‌کردن مقدار یک برای مرکزکردن تقریبی سیگنال حول صفر؛
 ۸. تقسیم سیگنال‌های طولانی به پنجره‌های 256 نمونه‌ای؛
 ۹. استفاده از گام 128 و ایجاد هم‌پوشانی 50 درصدی؛
 ۱۰. حذف پنجره‌های شامل برچسب صفر، زیرا این برچسب نشان‌دهنده‌ی داده‌های خارج از بخش اصلی آزمایش است؛
 ۱۱. استفاده از دنباله‌های نرمال برای آموزش مدل‌های تک‌کلاسه؛
 ۱۲. ساخت مجموعه‌ی آزمون با نسبت ناهنجاری 10 درصد؛
 ۱۳. استانداردسازی ویژگی‌ها با استفاده از *StandardScaler*.
- استانداردسازی فقط با استفاده از مجموعه‌ی آموزش برازش شده است:

$$\mu_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} X_{\text{train}}^{(i)}.$$

سپس همان میانگین و انحراف معیار به‌دست‌آمده از مجموعه‌ی آموزش، برای تبدیل داده‌های آموزش و آزمون استفاده شده‌اند:

$$X'_{\text{train}} = \frac{X_{\text{train}} - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

$$X'_{\text{test}} = \frac{X_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

در هیچ مرحله‌ای، استانداردساز به‌طور جداگانه روی داده‌های آزمون برازش نشده است. در نتیجه، اطلاعات آماری مجموعه‌ی آزمون وارد فرایند آموزش نشده و از نشت داده جلوگیری شده است.

۲.۱ ECG

معرفی مجموعه‌داده

در این پروژه از مجموعه‌داده‌ی *ECG5000* استفاده شده است. این مجموعه‌داده شامل سیگنال‌های الکتروکاردیوگرام یا *Electrocardiogram (ECG)* است و برای بررسی و تشخیص ضربان‌های طبیعی و غیرطبیعی قلب به کار می‌رود. داده‌ی اصلی از یک سیگنال تقریباً بیست‌ساعته‌ی *ECG* مربوط به یک بیمار مبتلا به نارسایی شدید قلبی استخراج شده است. این سیگنال مربوط به رکورد *chf07* از مجموعه‌داده‌ی *BIDMC Congestive Heart Failure Database* است. برای تولید مجموعه‌داده‌ی *ECG5000*، ابتدا ضربان‌های قلب از سیگنال اصلی استخراج شده‌اند. سپس طول ضربان‌ها با استفاده از درون‌یابی یکسان شده و در نهایت تعداد 5000 ضربان انتخاب شده‌اند. بنابراین هر نمونه نشان‌دهنده‌ی یک ضربان کامل قلب است. برچسب‌های اصلی مجموعه‌داده به‌صورت زیر تعریف شده‌اند:

- 1 = قلب طبیعی ضربان
- 2 = نوع از بطنی زودرس انقباض *R-on-T PVC*
- 3 = بطنی زودرس انقباض *PVC*
- 4 = فوق‌بطنی زودرس ضربان *SP*
- 5 = طبقه‌بندی‌نشده ضربان *UB*

منابع مجموعه‌داده

نسخه‌ی استفاده‌شده در پروژه:

Salsabil Ahmid. ECG5000 Dataset. Kaggle.

Kaggle – ECG5000 Dataset

منبع رسمی مجموعه‌داده‌ی پردازش‌شده:

UCR/UEA Time Series Classification Archive – ECG5000

منبع سیگنال اولیه:

Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. Ch., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C.-K., & Stanley, H. E. (2000).

PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals.

PhysioNet – BIDMC Congestive Heart Failure Database

ماهیت دنباله‌ای داده‌ها و دلیل انتخاب آن

هر نمونه در این مجموعه داده یک دنباله‌ی زمانی از دامنه‌ی سیگنال الکتریکی قلب در طول یک ضربان کامل است. مقادیر ثبت شده در زمان‌های متوالی مستقل از یکدیگر نیستند؛ زیرا شکل هر ضربان قلب از بخش‌های متوالی و وابسته به زمان تشکیل شده است.

موقعیت، دامنه، پهنا و ترتیب زمانی این بخش‌ها در تشخیص طبیعی یا غیرطبیعی بودن ضربان اهمیت دارند. بنابراین ممکن است دو دنباله دارای میانگین و واریانس مشابهی باشند، اما شکل و ترتیب زمانی متفاوتی داشته باشند. پس مجموعه داده‌ی *ECG5000* برای مقایسه‌ی مدل پایه‌ی *OC-SVM* با مدل آگاه از دنباله‌ی *HMAD* مناسب است؛ زیرا مدل *HMAD* می‌تواند علاوه بر مقادیر دامنه، انتقال میان حالت‌های پنهان مختلف در طول ضربان را نیز در نظر بگیرد.

ویژگی‌های خام مجموعه داده

جدول ۳: ساختار ویژگی‌های خام مجموعه داده‌ی *ECG5000*

توضیح	ویژگی	شماره ستون
عددی بین 1 تا 5 که نوع ضربان قلب را مشخص می‌کند	برچسب کلاس	0
140 مقدار متوالی از دامنه‌ی سیگنال یک ضربان قلب	دامنه‌ی سیگنال	1 تا 140

ویژگی استفاده شده در این پروژه

مجموعه داده‌ی *ECG5000* تک متغیره است؛ یعنی در هر گام زمانی تنها یک مقدار از دامنه‌ی سیگنال قلب ثبت شده است. بنابراین هر مشاهده یک مقدار اسکالر است: $x_t \in \mathbb{R}$. پس نیازی به آماده سازی برای اسکالر شدن ندارد. طول ضربان‌های استخراج شده در داده‌ی اولیه یکسان نبوده است، اما تولیدکنندگان مجموعه داده با استفاده از درون‌یابی، طول تمام ضربان‌ها را برابر با 140 قرار داده‌اند. در کد این پروژه هیچ درون‌یابی، پنجره‌بندی یا تغییر طول جدیدی انجام نشده است.

روش تعیین دنباله‌های نرمال و ناهنجار

مجموعه داده‌ی اصلی شامل پنج کلاس است. برای استفاده از آن در مسئله‌ی تشخیص ناهنجاری، برچسب‌ها به یک مسئله‌ی دودویی تبدیل شده‌اند. کلاس اول به عنوان نرمال در نظر گرفته شده است:

$$y_{\text{original}}^{(i)} = 1 \implies y^{(i)} = 0.$$

تمام کلاس‌های دیگر به عنوان ناهنجار تعریف شده‌اند:

$$y_{\text{original}}^{(i)} \in \{2, 3, 4, 5\} \implies y^{(i)} = 1.$$

در نتیجه:

$$y^{(i)} = \begin{cases} 0, & y_{\text{original}}^{(i)} = 1, \\ 1, & y_{\text{original}}^{(i)} \neq 1. \end{cases}$$

بنابراین:

- برچسب صفر نشان دهنده‌ی ضربان نرمال است؛
- برچسب یک نشان دهنده‌ی ضربان ناهنجار است.

روش تولید مجموعه‌های آموزش و آزمون

در این پروژه از تقسیم رسمی مجموعه داده‌ی $ECG5000$ استفاده شده است. فایل آموزش اصلی شامل 500 دنباله و فایل آزمون اصلی شامل 4500 دنباله است:

$$N_{\text{train,raw}} = 500, \quad N_{\text{test}} = 4500.$$

از آنجاکه مدل‌های $OC-SVM$ و $HMAD$ به صورت تک‌کلاسه آموزش داده می‌شوند، فقط دنباله‌های نرمال کلاس اول از فایل آموزش انتخاب شده‌اند:

$$X_{\text{train,normal}} = \left\{ \mathbf{x}^{(i)} \mid y_{\text{train,raw}}^{(i)} = 1 \right\}.$$

در فایل آموزش اصلی، تعداد 292 دنباله‌ی نرمال و 208 دنباله‌ی غیرنرمال وجود دارد. در نتیجه، تنها 292 دنباله‌ی نرمال برای آموزش استفاده شده‌اند و نمونه‌های غیرنرمال فایل آموزش کنار گذاشته شده‌اند. مجموعه‌ی آزمون شامل:

$$N_{\text{test,normal}} = 2627$$

دنباله‌ی نرمال و

$$N_{\text{test,anomaly}} = 1873$$

دنباله‌ی ناهنجار است. بنابراین نسبت واقعی ناهنجاری مجموعه‌ی آزمون برابر است با:

$$r_{\text{test}} = \frac{1873}{4500} \approx 0.4162.$$

یعنی تقریباً 41.62 درصد از دنباله‌های آزمون ناهنجار هستند.

تعداد دنباله‌ها و توزیع برجسب‌ها

جدول ۴: تعداد دنباله‌های نرمال و ناهنجار

مجموعه	نرمال	ناهنجار	مجموع
آموزش اصلی	292	208	500
آموزش استفاده شده	292	0	292
آزمون	2627	1873	4500
کل داده‌های استفاده شده	2919	1873	4792

بنابراین ماتریس‌های نهایی داده دارای ابعاد زیر هستند:

$$X_{\text{train}} \in \mathbb{R}^{292 \times 140},$$

$$X_{\text{test}} \in \mathbb{R}^{4500 \times 140}.$$

تمام دنباله‌های مجموعه‌ی آموزش نرمال هستند، درحالی‌که مجموعه‌ی آزمون شامل هر دو گروه نرمال و ناهنجار است. تمام دنباله‌ها دارای طول ثابت 140 هستند؛ بنابراین توزیع طول دنباله‌ها به صورت زیر است:

$$T_{\min} = T_{\max} = T_{\text{mean}} = T_{\text{median}} = 140,$$

$$\sigma_T = 0.$$

در نتیجه، مجموعه داده شامل دنباله‌های با طول متغیر نیست و تمام ورودی‌ها ابعاد یکسانی دارند.

مراحل پیش‌پردازش

مراحل پیش‌پردازش انجام شده طبق کد پروژه به ترتیب زیر هستند:

۱. خواندن فایل‌های متنی *ECG5000_TRAIN* و *ECG5000_TEST* با استفاده از جداکننده‌ی فاصله؛
 ۲. در نظر گرفتن ستون اول هر فایل به عنوان برچسب کلاس؛
 ۳. در نظر گرفتن 140 ستون بعدی به عنوان مقادیر دنباله‌ی زمانی؛
 ۴. تبدیل مقادیر دنباله‌ها به نوع عددی *float64*؛
 ۵. انتخاب فقط دنباله‌های دارای برچسب اصلی 1 از فایل آموزش؛
 ۶. حذف دنباله‌های غیرنرمال فایل آموزش از فرایند آموزش تک‌کلاسه؛
 ۷. نگه داشتن تمام دنباله‌های فایل آزمون بدون نمونه‌گیری مجدد؛
 ۸. تبدیل برچسب کلاس اول آزمون به نرمال و کلاس‌های دیگر به ناهنجار؛
 ۹. استفاده از دنباله‌ی کامل 140 بعدی به عنوان نمایش تخت‌شده‌ی مدل پایه؛
 ۱۰. استخراج ویژگی‌های خلاصه‌ی کلی شامل میانگین، انحراف معیار، کمینه، بیشینه و میانه برای آزمایش نمایش دوم؛
 ۱۱. تقسیم هر دنباله به چهار بخش 35 تایی و محاسبه‌ی میانگین و انحراف معیار هر بخش برای آزمایش نمایش سوم برای نمایش پنجره‌ی ای
 ۱۲. استانداردسازی هر نمایش ویژگی با استفاده از *StandardScaler*؛
- در نمایش تخت‌شده، استانداردسازی فقط با استفاده از داده‌های نرمال آموزش برآزش شده است:

$$\mu_{\text{train},j} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} X_{\text{train},ij}.$$

سپس همان میانگین و انحراف معیار برای تبدیل داده‌های آموزش و آزمون استفاده شده‌اند:

$$X'_{\text{train},ij} = \frac{X_{\text{train},ij} - \mu_{\text{train},j}}{\sigma_{\text{train},j}},$$

$$X'_{\text{test},ij} = \frac{X_{\text{test},ij} - \mu_{\text{train},j}}{\sigma_{\text{train},j}}.$$

در هیچ مرحله‌ای، استانداردسازی به‌طور جداگانه روی داده‌های آزمون برآزش نشده است. در نتیجه، اطلاعات آماری مجموعه‌ی آزمون وارد فرایند آموزش نشده و از نشت داده جلوگیری شده است.

۲ Introduction to Anomaly Detection

۱.۲ Informal Definition

Theoretical Question 1

Give one example of a point anomaly and one example of a sequence anomaly. Explain why the sequence anomaly may not be detectable by looking at each observation independently.

یک نمونه از ناهنجاری نقطه‌ای را می‌توان در اندازه‌گیری دمای یک دستگاه مشاهده کرد. برای مثال، دنباله

22, 23, 21, 24, 85, 22

را در نظر می‌گیریم. بیشتر مقادیر در محدوده طبیعی قرار دارند، اما مقدار 85 به‌تنهایی فاصله زیادی از سایر داده‌ها دارد؛ بنابراین حتی بدون توجه به ترتیب زمانی نیز می‌توان آن را یک ناهنجاری نقطه‌ای دانست. در ناهنجاری‌های دنباله‌ای اما ممکن است خود مقادیر تک‌تک نرمال بوده و در محدوده طبیعی باشند ولی دنباله آنها کنار یکدیگر نشان دهنده ناهنجاری باشند مثلاً اگر دمای یک دستگاه را در طی زمان اینگونه در نظر بگیریم:

22, 25, 40, 24, 22

ناهنجاری ناشی از این است که دستگاه در زمان کمی داغ شده و سپس در زمان کمی خنک شده ولی خود دمای 40 ممکن است در محدوده نرمال دمای دستگاه باشد ولی نه درحالی‌که که دمای آن کمی قبل‌تر 25 بوده و مثلاً دستگاه در حال استراحت است و کار نمی‌کند.

علت این موضوع آن است که بررسی مستقل هر مشاهده تنها احتمال حاشیه‌ای آن، یعنی $P(x_t)$ ، را در نظر می‌گیرد؛ درحالی‌که ناهنجاری دنباله‌ای به احتمال مشترک و انتقال‌های زمانی وابسته است. برای یک مدل مرتبه اول مارکوف داریم

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t | x_{t-1}).$$

بنابراین ممکن است تمام مقادیر $P(x_t)$ بزرگ باشند، اما یک یا چند احتمال انتقال $P(x_t | x_{t-1})$ بسیار کوچک باشد. در این حالت، هر مشاهده به‌تنهایی طبیعی به نظر می‌رسد، ولی کل دنباله غیرعادی است.

۲.۲ Formal Problem Statement

Theoretical Question 2

Why is anomaly detection harder than ordinary binary classification when labeled anomalous examples are unavailable or unreliable?

تشخیص ناهنجاری‌ها در این حالت سخت‌تر از طبقه‌بندی دودویی است چون در طبقه‌بندی دودویی ما لیبل هر دو کلاس را داریم و داده‌های train برای هر دو کلاس موجود هستند چه نرمال چه ناهنجار و مدل مرز بین این دو کلاس را یاد می‌گیرد ولی در مدل OC-SVM ما داده‌ها را برای ناهنجاری بسیار کم داریم و کمیاب هستند و برچسب آنها نیز قابل اعتماد نیست و یا هزینه بر است پس مدل مستقیماً ناهنجاری‌ها را یاد نمی‌گیرد و به وسیله یادگیری داده‌های نرمال

ناهنجاری را تشخیص میدهد. یعنی مدل بین دو کلای مرز نمیکشد بلکه داده های نرمال را دیده و رفتار نرمال را یاد میگیرد و سپس با مقایسه بررسی میکند که با الگو های نرمال سازگار هست یا نه و هر نمونه ای که با الگویی که یاد گرفته سازگار نیست را ناهنجار تشخیص میدهد. این کار دشوار است چون داده های ناهنجار ممکن است بسیار شبیه داده های نرمال باشند.

۳.۲ Density Level Set View

Theoretical Question 3

Suppose $\nu = 0.05$. What does this say about the amount of probability mass that the ideal detector is allowed to place outside the nominal region?

این یعنی مدل ایده آل اجازه دارد حدود ۵ درصد از جرم احتمال توزیع نرمال را برای داده های خارج ناحیه یا ناهنجاری ها در نظر بگیرد و مثلاً حدود ۵ درصد از داده ها که در ناحیه کم چگال تر است را بیرون ناحیه داده های نرمال در نظر بگیرد و ۹۵ درصد باقی مانده را داده های پرتکرار و نرمال را در بر بگیرند. به طور کلی ν نشان دهنده مقدار مجاز داده هایی است که میتوانند به عنوان ناهنجاری در نظر گرفته شوند. به بیان ریاضی:

$$P(X \notin R_\nu) \leq \nu = 0.05,$$

یا به طور معادل،

$$P(X \in R_\nu) \geq 1 - \nu = 0.95.$$

۴.۲ Connection to This Project

۳ OC-SVM Baseline

۱.۳ Why OC-SVM?

Theoretical Question 4

Why is OC-SVM called a “one-class” method? Explain how its training setting differs from ordinary binary classification.

به این دلیل تک کلاسه میگویند زیرا مدل فقط نمونه های مربوط به کلاس نرمال را آموزش میبندد و برخلاف طبقه بندی دودویی که از هر دو کلاس داده میبندد روی هر دو کلاس ترین میشود و از هر دو کلاس برچسب هایشان را در اختیار دارد در این مدل برچسب ناهنجاری وجود ندارد یا قابل اعتماد نیست پس هدف در اینجا کشیدن مرز بین دو کلاس نیست ولی در طبقه بندی دودویی هدف پیدا کردن مرز برای دو کلاس بود بلکه اینجا مدل تلاش میکند که داده های نرمال را یاد گرفته و سپس هر نمونه که به آن بدهیم مقایسه کرده و تشخیص دهد با الگو نرمالی که آموزش دیده سازگار است یا خیر و اگر نبود خارج این ناحیه دسته بندی میشود که به عنوان ناهنجاری شناخته شود.

Theoretical Question 5

Why is OC-SVM a meaningful baseline for this project, even though the project later studies sequence-aware anomaly detection?

این مدل از روش های پرکاربرد برای تشخیص ناهنجاری تک کلاسه مبتنی بر روش های کرنل است و برای وقتی که داده ها عمدتاً نرمال باشند و برجسب ناهنجاری ها در دست نباشد مفید است ولی اثر دنباله را در نظر نمیگیرد و یک بردار ویژگی با طول ثابت در نظر میگیرد و ترتیب زمانی و حالت پنهان وابستگی هارا نمیبیند برای همین از آن استفاده کرده و سپس با مدلی که در ادامه معرفی میشود مقایسه میکنیم تا اثر این موضوع را بررسی کنیم. در این صورت میتوانیم بررسی کنیم آیا این روش عمل میکند یا خیر و مدل سازی واقعا ساختار دنباله و ترتیب زمانی را درست مدل میکند یا خیر.

۲.۳ Model and Objective

Theoretical Question 6

What is the geometric meaning of separating the mapped training data from the origin in the OC-SVM feature space?

در این روش داده ها را ابتدا به وسیله یک فیچر مپ به فضا با ابعاد بالاتری میبریم که فضای ویژگی های ما هستند و در این فضا سعی میکند ابرصفحه ای پیدا کند که داده های نرمال را بهتر از مبدا جدا کند یعنی مدل در فضای ویژگی ها ابرصفحه ای را در نظر میگیرد که مانند مرز عمل کند و نمونه ها را در یک سمت آن نرمال و در طرف دیگر ناهنجاری در نظر بگیرد و در واقع به جای جدا کردن دو کلاس تلاش میکند داده های نرمال را از مبدا جدا کند و یک مرز اطراف بخش پرتراکم داده های نرمال بسازد و هر داده هایی که نتواند در این ناحیه قرار دهد را به عنوان ناهنجاری احتمالی در نظر بگیرد. به بیان ریاضی ابر صفحه ای که مدل در فضای ویژگی ها در نظر میگیرد :

$$\langle w, \phi(x) \rangle - \rho = 0$$

تا داده های آموزشی نرمال را تا حد امکان از مبدا جدا کند و فاصله میان ابرصفحه و مبدا را بزرگ نگه دارد. برای یک نمونه x ، تابع تصمیم برابر است با

$$f(x) = \langle w, \phi(x) \rangle - \rho.$$

اگر $f(x) \geq 0$ باشد، نمونه در سمت نرمال ابرصفحه قرار میگیرد و اگر $f(x) < 0$ باشد، نمونه ناهنجار یا مشکوک در نظر گرفته می شود.

از نظر هندسی، مدل به جای جدا کردن دو کلاس برجسب خورده، ناحیه ای را در فضای ویژگی مشخص می کند که بیشتر داده های نرمال در آن قرار دارند. به کمک کرنل، تصویر این ابرصفحه در فضای ورودی می تواند یک مرز غیرخطی پیرامون داده های نرمال ایجاد کند.

Theoretical Question 7

Why are slack variables ξ_i needed in the OC-SVM objective? Relate your answer to noisy or contaminated training data.

با توجه به ریاضی سوال: قید مربوط به هر نمونه به شکل

$$\langle w, \phi(x_i) \rangle \geq \rho - \xi_i, \quad \xi_i \geq 0$$

است. اگر $\xi_i = 0$ باشد، نمونه قید اصلی را رعایت می کند. اگر $\xi_i > 0$ باشد، مدل اجازه می دهد آن نمونه به اندازه ξ_i از مرز عبور کند یا در سمت ناهنجار قرار گیرد.

وجود متغیر های اسلک ضروری است زیرا داده های واقعی ممکن است شامل داده های نرمال و بدون نویز نباشد و تعداد کمی نویز هم داشته باشیم در این حالت مدل اگه بخواهد مرزی در نظر بگیرد که آن داده های نویزی را نیز

شامل شود باید مرزی پیچیده بکشد و به این داده های نویزی حساس شود ولی وجود این متغیر های اسلک به مدل این اجازه را میدهد که برخی نمونه هایی که میبیند را خارج از ناحیه مورد نظر در نظر بگیرد و اجازه دهد مرز را رد کنند تا مدل نسبت به نویز مقاوم تر شود و مرز ساده تری در نظر بگیرد و به جای تطبیق خود با داده های نویزی الگوی کلی داده های نرمال را یاد بگیرد و در واقع این متغیر ها نوعی انعطاف پذیری به مدل میدهند که بین داده هایی که اجازه میدهد خارج مرز قرار بگیرند و مرز ساده تری بکشد و اورفیت هم نکند تریداف داشته باشد زیرا در تابع هدف مجموع متغیر های اسلک را جریمه میکنیم:

$$\frac{1}{\nu n} \sum_{i=1}^n \xi_i.$$

۳.۳ Interpretation of ν

Theoretical Question 8

Explain the role of ν in OC-SVM. What may happen if ν is chosen much smaller or much larger than the true contamination level?

پارامتر ν یک هایپر پارامتر است که ما ابتدا به مدل میدهم تا میزان سخت گیری مدل را نسبت به داده های آموزشی کنترل کنیم و مشخص میکند مدل تا چه اندازه اجازه دارد بخشی از داده های آموزشی را خارج از ناحیه نرمال در نظر گرفت. همچنین ν حد پایینی برای بردار های ساپورت و حد بالایی برای درصد داده های پرت در نظر میگیرد. چراکه: مسئله اولیه OC-SVM برابر است با

$$\begin{aligned} \min_{w, \rho, \xi} \quad & \frac{1}{2} \|w\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^n \xi_i, \\ \text{to subject} \quad & \langle w, \phi(x_i) \rangle \geq \rho - \xi_i, \quad i = 1, \dots, n, \\ & \xi_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

برای قید

$$\langle w, \phi(x_i) \rangle - \rho + \xi_i \geq 0$$

ضریب لاگرانژ $\alpha_i \geq 0$ و برای قید $\xi_i \geq 0$ ضریب $\beta_i \geq 0$ در نظر میگیریم. تابع لاگرانژ به صورت زیر است:

$$\begin{aligned} \mathcal{L} = & \frac{1}{2} \|w\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^n \xi_i \\ & - \sum_{i=1}^n \alpha_i (\langle w, \phi(x_i) \rangle - \rho + \xi_i) - \sum_{i=1}^n \beta_i \xi_i. \end{aligned}$$

از شرط ایستایی نسبت به ρ داریم

$$\frac{\partial \mathcal{L}}{\partial \rho} = 0 \implies -1 + \sum_{i=1}^n \alpha_i = 0,$$

بنابراین

$$\sum_{i=1}^n \alpha_i = 1.$$

همچنین، از شرط ایستایی نسبت به ξ_i به دست می‌آید:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies \frac{1}{\nu n} - \alpha_i - \beta_i = 0.$$

از آنجا که $\beta_i \geq 0$ ، نتیجه می‌شود

$$0 \leq \alpha_i \leq \frac{1}{\nu n}.$$

اکنون تعداد بردارهای پشتیبان را با m نشان می‌دهیم. هر نمونه‌ای که $\alpha_i > 0$ داشته باشد، بردار پشتیبان است. از آنجا که فقط بردارهای پشتیبان در مجموع ضرایب سهم دارند، داریم

$$1 = \sum_{i=1}^n \alpha_i \leq m \left(\frac{1}{\nu n} \right).$$

در نتیجه،

$$m \geq \nu n \implies \frac{m}{n} \geq \nu.$$

پس ν حد پایینی برای نسبت بردارهای پشتیبان است. حال تعداد نمونه‌های آموزشی واقع شده در سمت ناهنجار مرز را با q نشان می‌دهیم. برای چنین نمونه‌ای داریم

$$f(x_i) = \langle w, \phi(x_i) \rangle - \rho < 0.$$

بنابراین برای برقرار ماندن قید باید $\xi_i > 0$ باشد. شرط مکملی KKT برای قید نامنفی بودن اسلک برابر است با

$$\beta_i \xi_i = 0.$$

چون $\xi_i > 0$ ، نتیجه می‌شود $\beta_i = 0$. از رابطه

$$\alpha_i + \beta_i = \frac{1}{\nu n}$$

داریم

$$\alpha_i = \frac{1}{\nu n}.$$

در نتیجه، هر نمونه آموزشی واقع شده در سمت ناهنجار مرز، مقدار $\frac{1}{\nu n}$ از مجموع ضرایب α_i را به خود اختصاص می‌دهد. بنابراین

$$1 = \sum_{i=1}^n \alpha_i \geq q \left(\frac{1}{\nu n} \right),$$

و در نتیجه

$$q \leq \nu n \implies \frac{q}{n} \leq \nu.$$

پس در حل استاندارد $OC-SVM$ رابطه زیر برقرار است:

$$\boxed{\frac{\text{تعداد خطاهای آموزشی}}{n} \leq \nu \leq \frac{\text{تعداد بردارهای پشتیبان}}{n}}$$

اگر ν بسیار کوچک‌تر از سطح واقعی آلودگی داده‌ها انتخاب شود، مدل بیش از حد محافظه‌کار می‌شود و تمایل کمی به رد کردن نمونه‌های آموزشی دارد. در این حالت، ناحیه‌ی نرمال بیش از حد بزرگ می‌شود و ممکن است تعدادی از نمونه‌های

آلوده یا ناهنجار نیز داخل ناحیه‌ی نرمال قرار بگیرند. در نتیجه، مدل بخشی از ناهنجاری‌ها را به‌عنوان داده‌ی نرمال یاد می‌گیرد و در زمان آزمون ممکن است ناهنجاری‌ها را به‌خوبی تشخیص ندهد. در مقابل، اگر ν بسیار بزرگ‌تر از سطح واقعی آلودگی انتخاب شود، مدل بیش از حد سخت‌گیر می‌شود و اجازه می‌دهد تعداد زیادی از نمونه‌های آموزشی خارج از ناحیه‌ی نرمال قرار بگیرند. در این حالت، ناحیه‌ی نرمال بیش از حد کوچک و محدود می‌شود و حتی برخی نمونه‌های سالم و نرمال نیز ممکن است به‌اشتباه ناهنجار تشخیص داده شوند. این موضوع باعث افزایش خطاهای مثبت کاذب و کاهش پایداری مدل می‌شود. در واقعیت چون مقدار ν هاپیر پارامتر است و خودمان انتخاب می‌کنیم ولی اطلاع دقیقی از آن نداریم برای شبیه‌سازی‌ها از چند مقدار برای ν استفاده می‌کنیم.

۴.۳ Implementation Requirements

Theoretical Question 9

Why does converting a sequence into a fixed-length vector not necessarily mean that temporal dependency has been properly modeled?

تبدیل یک دنباله به یک بردار با طول ثابت فقط باعث می‌شود داده برای مدل‌هایی مثل OC-SVM قابل استفاده شود، اما الزاماً به این معنا نیست که رابطه‌ی زمانی بین مشاهدات به‌درستی در مدل وارد شده است. در یک دنباله، ترتیب، فاصله‌ی زمانی، تغییرات پشت‌سرهم و الگوی انتقال از یک مقدار به مقدار بعدی اهمیت دارند. اما وقتی دنباله را فقط به چند ویژگی خلاصه مانند میانگین، انحراف معیار، کمینه و بیشینه تبدیل می‌کنیم، بسیاری از این اطلاعات زمانی از بین می‌روند. حتی در روش flatten کردن نیز اگرچه ترتیب مقادیر تا حدی حفظ می‌شود، اما رابطه‌ی ای مثل

$$P(x_{t+1} | x_t)$$

یا وابستگی پنهان بین آنها را یاد نمی‌گیرد و در نهایت مدل در فضای ویژگی‌ها فقط شباهت و فاصله بردار ویژگی‌ها را بررسی می‌کند. برای مثال، دو دنباله ممکن است میانگین، واریانس، کمینه و بیشینه‌ی یکسانی داشته باشند، اما ترتیب مقادیر در آن‌ها کاملاً متفاوت باشد. یکی ممکن است روندی طبیعی و آرام داشته باشد، در حالی که دیگری شامل پرش‌های ناگهانی و غیرعادی باشد. اگر نمایش برداری این تفاوت ترتیبی را خوب ثبت نکند، مدل نمی‌تواند ناهنجاری دنباله‌ای را تشخیص دهد.

Simulation Question 1

Implement an OC-SVM baseline using scikit-learn on the selected dataset. Report the feature representation, preprocessing, kernel, ν , kernel parameters, and ROC-AUC.

در این آزمایش، یک مدل پایه *One-Class SVM* برای هر دو دیتاست آموزش داده شد. هدف، بررسی عملکرد یک روش پایه بدون استفاده از اطلاعات ترتیبی دنباله‌ها در تشخیص ناهنجاری است. در هر دو دیتاست، نمایش ویژگی مورد استفاده *Flattened sequence* بود. به این معنی که هر دنباله زمانی بدون استخراج ویژگی اضافی، به صورت یک بردار ویژگی به مدل داده شد. تنظیمات مدل برای دو دیتاست در جدول زیر گزارش شده است.

جدول ۵: OC-SVM configuration for two datasets

Parameter	ECG5000	Daphnet FOG
Normal training sequences	292	600
Sequence length	140	256
Feature representation	Flattened sequence	Flattened sequence
Feature dimension	140	256
Preprocessing	StandardScaler	StandardScaler
Kernel	RBF	RBF
ν	0.1	0.1
γ	scale	scale

پس از آموزش مدل، خروجی *decision function* به عنوان امتیاز نرمال بودن نمونه‌ها استفاده شد. از آنجا که مقدار بزرگ‌تر این تابع نشان‌دهنده شباهت بیشتر به داده‌های نرمال است، برای محاسبه *ROC-AUC* علامت آن تغییر داده شد تا مقدار بزرگ‌تر نشان‌دهنده احتمال بیشتر ناهنجاری باشد. نتایج ارزیابی مدل OC-SVM بر روی داده‌های آزمون در جدول زیر ارائه شده است.

جدول ۶: OC-SVM performance comparison

Metric	ECG5000	Daphnet FOG
ROC-AUC	0.984025	0.375309
Precision	0.001776	0.250000
Recall	0.002136	0.050000
F1-score	0.001939	0.083333

نتایج نشان می‌دهد که عملکرد مدل پایه OC-SVM بین دو دیتاست تفاوت قابل توجهی دارد. در دیتاست ECG5000 مقدار *ROC-AUC* برابر با 0.984025 است که نشان‌دهنده توانایی مناسب مدل در رتبه‌بندی نمونه‌های نرمال و ناهنجار است.

با این حال، مقادیر *Precision* و *Recall* پایین هستند که نشان می‌دهد آستانه پیش‌فرض مدل در تبدیل امتیازها به برچسب نهایی، تعداد زیادی خطای تشخیص ایجاد می‌کند. در دیتاست Daphnet FOG مقدار *ROC-AUC* برابر با 0.375309 به دست آمده است. این مقدار پایین نشان می‌دهد که نمایش ساده *Flattened sequence* برای ثبت ساختار زمانی پیچیده این داده کافی نیست. این نتایج نشان می‌دهد که استفاده از مقادیر خام دنباله بدون در نظر گرفتن وابستگی‌های زمانی، محدودیت‌هایی دارد و استفاده از روش‌هایی که ساختار ترتیبی داده‌ها را مدل می‌کنند، مانند HMAD می‌تواند عملکرد تشخیص ناهنجاری را بهبود دهد.

Simulation Question 2

Compare at least two sequence feature representations for OC-SVM, such as flattened features and summary or window-based features. Which representation works better, and why?

در این شبیه‌سازی، تأثیر نوع نمایش ویژگی بر عملکرد مدل OC-SVM بررسی شد. هدف این آزمایش مقایسه روش‌های مختلف تبدیل یک دنباله زمانی به فضای ویژگی و بررسی تأثیر آن‌ها بر تشخیص ناهنجاری است. برای این منظور سه نمایش مختلف از دنباله‌ها مورد بررسی قرار گرفت:

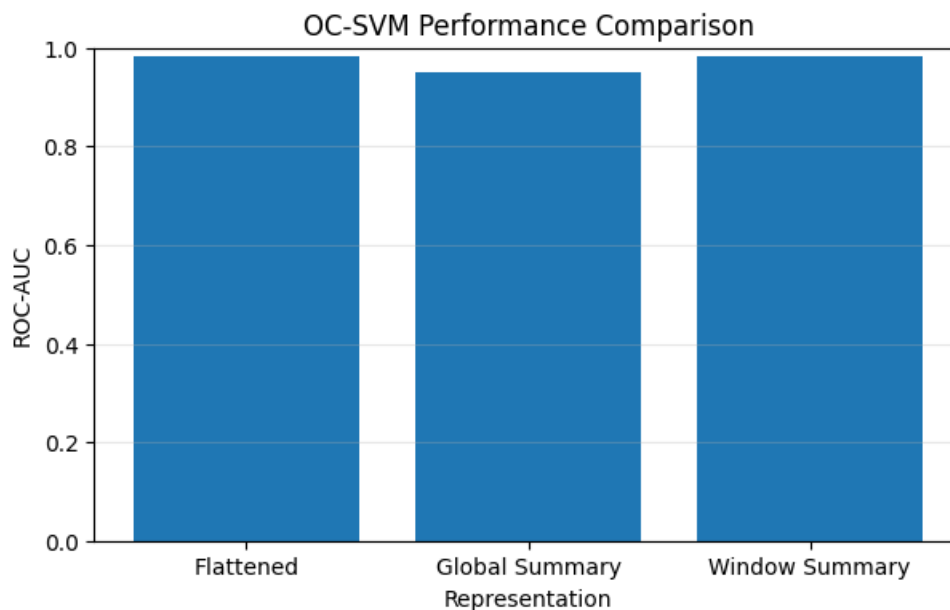
- *Flattened Sequence*: در این روش، تمام نقاط زمانی دنباله به صورت مستقیم به عنوان ویژگی ورودی مدل استفاده شدند.

- *Global Summary*: در این روش، ویژگی‌های آماری کلی شامل میانگین، انحراف معیار، کمینه، بیشینه و میانه از هر دنباله استخراج شدند.
 - *Window Summary*: در این روش، دنباله به چند پنجره تقسیم شده و ویژگی‌های آماری هر پنجره به صورت جداگانه محاسبه شدند.
- ابعاد ویژگی ایجادشده توسط هر روش و مقدار *ROC-AUC* به عنوان معیار ارزیابی در جدول زیر گزارش شده است.

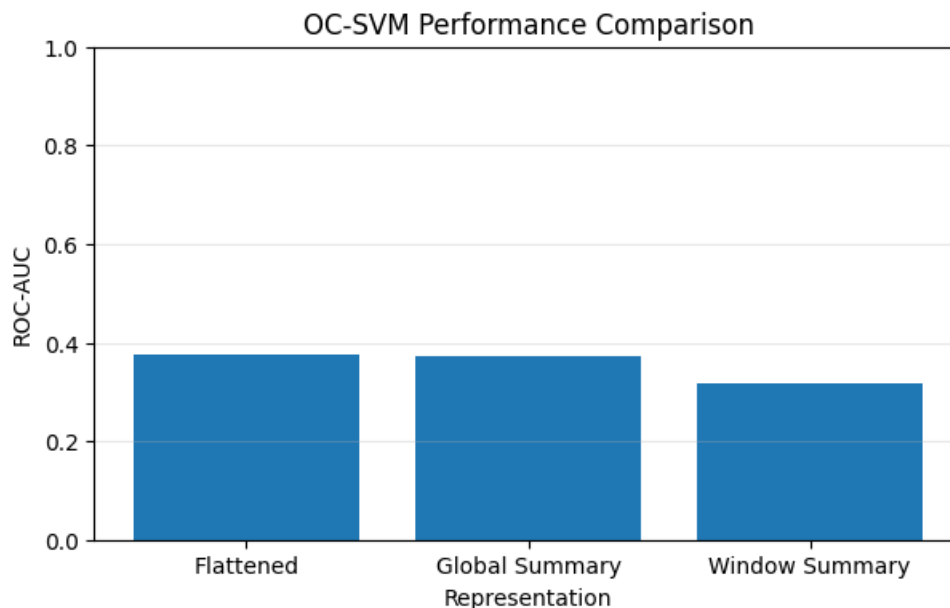
جدول ۷: Comparison of sequence feature representations

Dataset	Representation	Feature Dimension	ROC-AUC
ECG5000	Flattened Sequence	140	0.984025
ECG5000	Global Summary	5	0.951839
ECG5000	Window Summary	8	0.982451
Daphnet FOG	Flattened Sequence	256	0.375309
Daphnet FOG	Global Summary	5	0.372130
Daphnet FOG	Window Summary	32	0.317037

نتایج به دست آمده در شکل زیر مقایسه شده‌اند.



شکل ۱: OC-SVM performance comparison for different feature representations in ECG5000.



شکل ۲: OC-SVM performance comparison for different feature representations in Daphnet FOG.

نتایج نشان می‌دهد که تأثیر نوع نمایش ویژگی به ساختار دیتاست وابسته است. در دیتاست *ECG5000*، نمایش *Flattened Sequence* بهترین عملکرد را ارائه داد. همچنین روش *Window Summary* عملکرد بسیار نزدیکی داشت که نشان می‌دهد این دیتاست دارای الگوهای زمانی مشخصی است و حفظ اطلاعات جزئی دنباله برای تشخیص ناهنجاری مفید است. در مقابل، روش *Global Summary* با وجود کاهش شدید ابعاد ویژگی، عملکرد پایین‌تری داشت زیرا اطلاعات مربوط به تغییرات زمانی داخل دنباله حذف می‌شوند. در دیتاست *Daphnet FOG*، تمام روش‌ها عملکرد پایین‌تری نسبت به *ECG5000* داشتند. بهترین نتیجه مربوط به *Flattened Sequence* بود، در حالی که *Global Summary* مقدار 0.372130 و *Window Summary* مقدار 0.317037 را به دست آوردند. این نتیجه نشان می‌دهد که در دیتاست *Daphnet FOG* نمایش ساده ویژگی‌ها برای مدل *OC-SVM* کافی نیست و احتمالاً ناهنجاری‌ها دارای ساختار زمانی پیچیده‌تری هستند که با روش‌های مبتنی بر ویژگی خام یا آماری به خوبی قابل استخراج نیستند. به طور کلی، اگرچه *Flattened Sequence* در هر دو دیتاست بهترین نتیجه را ارائه داد، اما تفاوت زیاد عملکرد بین دو دیتاست نشان می‌دهد که انتخاب نمایش ویژگی باید با توجه به ساختار زمانی داده انجام شود.

Simulation Question 3

Evaluate OC-SVM for several values of ν , for example

$$\{0.01, 0.05, 0.1, 0.2\}.$$

Report how ROC-AUC changes and briefly interpret the result.

در این شبیه‌سازی، تأثیر پارامتر ν در مدل *One-Class SVM* بر عملکرد تشخیص ناهنجاری بررسی شد. پارامتر ν نقش مهمی در تعیین مرز تصمیم مدل دارد و به صورت مستقیم میزان حساسیت مدل نسبت به نمونه‌های خارج از توزیع نرمال و تعداد *Support Vector* ها را تحت تأثیر قرار می‌دهد. برای بررسی این اثر، مدل *OC-SVM* با مقادیر مختلف:

$$\nu \in \{0.01, 0.05, 0.1, 0.2\}$$

آموزش داده شد. در تمام آزمایش‌ها سایر پارامترهای مدل ثابت نگه داشته شدند و عملکرد مدل با معیار $ROC-AUC$ ارزیابی شد. همچنین تعداد $Support Vector$ های مدل برای بررسی تغییر پیچیدگی مرز تصمیم گزارش گردید. نتایج حاصل برای دو دیتاست در جدول‌های زیر ارائه شده است.

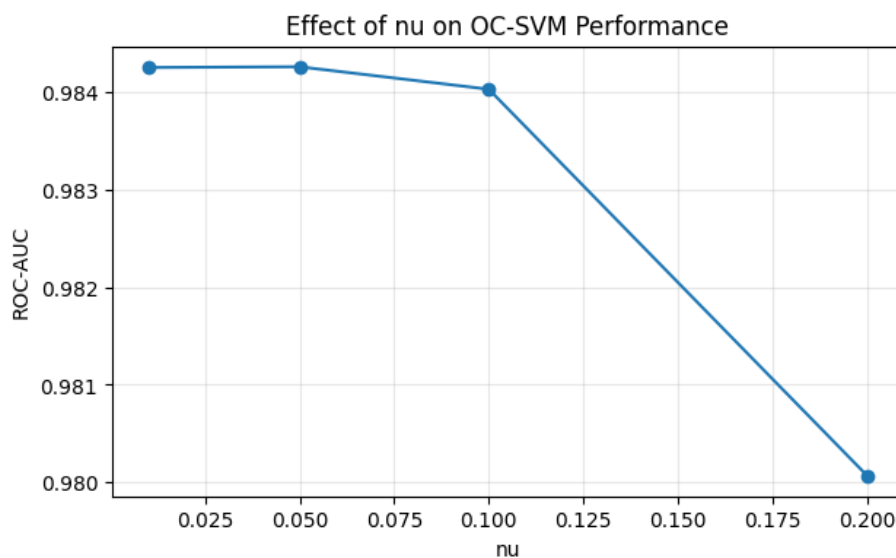
جدول ۸: $Effect of \nu$ on OC-SVM performance for ECG5000

ν	ROC-AUC	Number of Support Vectors
0.01	0.984247	54
0.05	0.984254	54
0.10	0.984025	54
0.20	0.980069	68

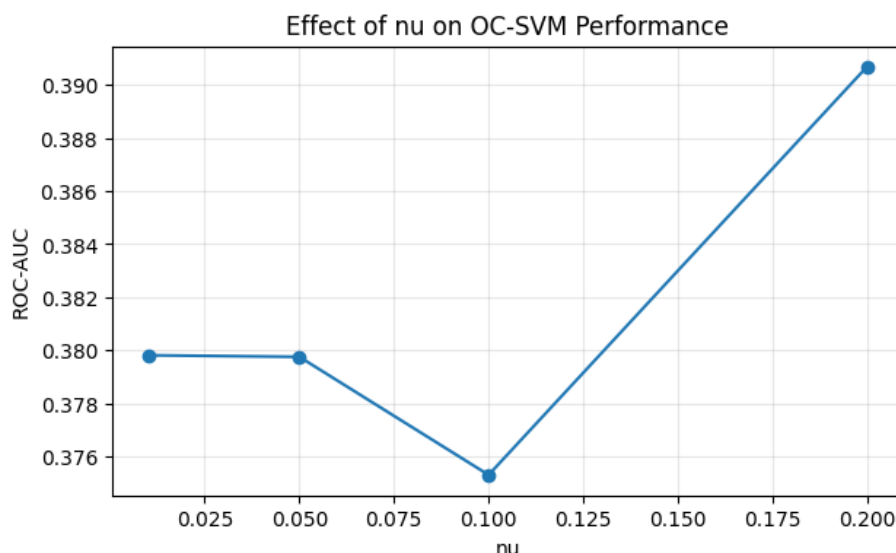
جدول ۹: $Effect of \nu$ on OC-SVM performance for Daphnet FOG

ν	ROC-AUC	Number of Support Vector
0.01	0.379815	85
0.05	0.379753	84
0.10	0.375309	92
0.20	0.390679	129

شکل زیر تغییرات مقدار $ROC-AUC$ بر حسب پارامتر ν را نمایش می‌دهد.



شکل ۳: $Effect of \nu$ on OC-SVM ROC-AUC performance in ECG5000



شکل ۴: Effect of ν on OC-SVM ROC-AUC performance in Daphnet FOG.

نتایج نشان می‌دهد که در هر دو دیتا ست تغییر مقدار ν تأثیر بسیار کمی بر عملکرد مدل دارد. مقدار ROC-AUC برای مقادیر مختلف تقریباً ثابت باقی مانده بهترین عملکرد در دیتا ست اول مربوط به

$$\nu = 0.05$$

است.

در دیتا ست دوم نیز بهترین عملکرد مربوط به

$$\nu = 0.2$$

است. بهترین مقدار افزایش ν باعث افزایش تعداد *Support Vector* ها شده که نشان‌دهنده پیچیده‌تر شدن مرز تصمیم مدل است. به طور کلی، انتخاب مقدار مناسب ν به ساختار داده وابسته است و مقدار بزرگ‌تر لزوماً باعث بهبود عملکرد نمی‌شود. بنابراین این پارامتر باید با استفاده از ارزیابی روی داده‌های اعتبارسنجی انتخاب شود.

۴ Latent Dependency and Real-World Motivation

۱.۴ Why the i.i.d. Assumption Can Fail

Theoretical Question 10

Explain why a detector that only looks at individual observations may fail to detect a sequence anomaly. Give one example where each individual observation is normal, but the ordering is suspicious.

مدلی که هر مشاهده را به صورت مستقل بررسی میکند فقط توجه میکند که این مقدار فعلی که در حال بررسی آن است به صورت مستقل نرمال است یا خیر و به مشاهدات قبلی و ترتیب زمانی رخدادها توجهی ندارد پس اگر ناهنجاری در دنباله و ترتیب زمانی رخدادها باشد متوجه آن نمیشود و مثلاً اگر تغییرات آب و هوایی را در نظر بگیریم احتمال رخداد برف بعد از هوای گرم افتابی کم است و اگر سنسوری تشخیص دهد پس از هوای گرم افتابی برف خواهد بارید پس سنسور دچار خطا شده ولی در این روش چون به بررسی مستقل هر داده میپردازیم خود پدیده برف باریدن طبیعی است و در نتیجه

ناهنجاری را گزارش نمیدهد ولی ترتیب زمانی که سنسور برای اب و هوا تشخیص داده که هوای گرم افتابی و سپس برف است ناهنجاری است پس این مدل ناهنجاری دنباله را اینگونه تشخیص نمیدهد.

Theoretical Question 11

For the server-log example above, explain why the sequence Port Scan $\rightarrow$ Login $\rightarrow$ Data Download could have low probability under a nominal transition model, even if each event has high marginal probability.

در واقع در مدلی که به بررسی هر رخداد به صورت مستقل میپردازد ما به بررسی احتمال های زیر میپردازیم :

$$P(\text{Scan Port}), P(\text{Login}), P(\text{Download Data})$$

و خود این احتمال ها ممکن است بالا باشند و در شرایط عادی رخ دهند پس احتمال حاشیه ای این رخداد ها میتواند زیاد باشد. ولی برای مدلی که ترتیب زمانی و دنباله ای در نظر میگیرد در واقعا احتمالی که بررسی میکند احتمال انتقال از یک رخداد به رخداد دیگر است یعنی احتمال شرطی را بررسی میکند برای مثال:

$$P(\text{Login}|\text{PortScan}), P(\text{Download Data}|\text{Login})$$

و درواقع برای دنباله مورد نظر داریم:

$$P(\text{Scan Port} \rightarrow \text{Login} \rightarrow \text{Download Data})$$

$$P(\text{Scan Port}), P(\text{Login} | \text{Scan Port}), P(\text{Download Data} | \text{Login}).$$

پس حتی اگر رخداد هر رویداد به تنهایی زیاد هم باشد ممکن است احتمال رخداد انتقال این رویداد ها به یکدیگر پایین شود و در نتیجه ناهنجاری در نظر گرفته شود چون در این حالت حاصل ضرب چند احتمال مجاسبه میشود و چون این احتمال ها عددی بین ۰ و ۱ هستند پس حاصل ضرب آنها مقداری کوچک میشود و میتواند احتمال کل این دنباله را خیلی پایین آورد.

برای همین در این روش چون به دنبال محاسبه زنجیر مارکوف و ترتیب خاص رویداد ها هستیم ممکن است با احتمال بالا برای هر رویداد به تنهایی در نهایت به احتمالی بسیار پایین برای دنباله ان رویداد ها برسیم به دلیل ضرب چندین احتمال در یکدیگر که احتمال نهایی دنباله را بسیار کوچک کند.

۲.۴ Markov Chains

Theoretical Question 12

Consider a two-state Markov chain with states 1 = normal and 2 = high-load. What does a large value of A_{11} mean? What does a large value of A_{12} mean?

در زنجیر مارکوف دو حالت :

$$A_{ij} = P(z_t = j | z_{t-1} = i).$$

پس A_{11} احتمال باقی ماندن سیستم در همان حالت نرمال است و اگر این مقدار بالا باشد یعنی سیستم اگر در زمان قبلی در حالت نرمال باشد به احتمال خیلی زیاد در زمان بعدی نیز نرمال خواهد بود و احتمال تغییر انکم است یعنی سیستم پایدار است و وضعیت نرمال خود را حفظ میکند.

حالا A_{12} یعنی احتمالا اینکه سیستم از حالت نرمال به حالت بار زیاد برود اگر این احتمال زیاد باشد یعنی درون سیستم انتقال از حالت نرمال به بار زیاد رایج است و مدام تکرار میشود.
حالا چون $A_{11} + A_{12} = 1$ پس این دو همزمان بزرگ نمیشوند یعنی سیستم یا دوست دارد حالت فلهی نرمال خود را حفظ کند یا تمایل زیادی به خروج از حالت نرمال به بار زیاد دارد.

Simulation Question 4

Write a Python function that computes the log-probability of a state sequence

$$z = (z_1, \dots, z_T)$$

under a given initial distribution π and transition matrix A :

$$\log P(z \mid \pi, A) = \log \pi_{z_1} + \sum_{t=2}^T \log A_{z_{t-1}, z_t}.$$

Use log-probabilities rather than raw products to avoid numerical underflow.

برای بررسی عملکرد تابع پیاده‌سازی‌شده، یک زنجیره مارکوف دو حالت با توزیع اولیه و ماتریس انتقال

$$\pi = [0.8, 0.2] \quad A = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}$$

داریم. مسیر حالت انتخاب‌شده نیز برابر با

$$z = [1, 1, 2, 2, 1]$$

بود که نشان‌دهنده انتقال‌های

$$1 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 1$$

است.

خروجی اجرای تابع `markov_log_probability` به صورت زیر به دست آمد:

```
sequence: State[1 1 2 2 1]
probability: Log - 5.678114727907623
Probability:0.003420000000000001
```

برای بررسی درستی تابع، احتمال مسیر به صورت مستقیم محاسبه شد:

$$P(z \mid \pi, A) = \pi_1 A_{11} A_{12} A_{22} A_{21} = (0.8)(0.95)(0.05)(0.90)(0.10) = 0.00342$$

همچنین مقدار لگاریتم احتمال برابر است با:

$$\log(0.00342) = -5.6781$$

که با مقدار خروجی تابع تطابق دارد. بنابراین تابع پیاده‌سازی‌شده توانسته است درست عمل کند

Hidden Markov Models ۳.۴

Theoretical Question 13

What is the difference between a Markov chain and a hidden Markov model? In your answer, distinguish between hidden states z_t and observations x_t .

در زنجیره مارکوف مدل مستقیماً حالت های سیستم را مشاهده میکند یعنی در هر زمان Z_t را میداند و میداند بین کدام حالت ها تغییر میدهیم و زنجیره مارکوف رو بر اساس حالت ها مینویسد و احتمال شرطی حساب میکند: $P(Z_t | Z_{t+1})$ را حساب میکند ولی در مدل مارکوف پنهان حالت واقعی سیستم را نداریم و در هر زمان اطلاعاتی یا اندازه گیری هایی مثل x_t داریم که با استفاده از آنها میتوانیم حالت سیستم را تشخیص دهیم و حالت پنهان ان را بیابیم مثل همون مثالی که مثلاً دمای ۸۰ درجه برای سیستم میتواند نشان دهنده حالت بار زیاد برای سیستم باشد و اینجا $X_t = 80$ ولی $Z_t = High-Load$ است و در واقعاً اطلاعاتی که ما داریم از زمان حاضر یک سری اندازه گیری ها هستند از مشاهدات ما در این حالت که با ان میتوان حالت فعلی سیستم را تخمین زد. برای همین دو نوع احتمال در این مدل محاسبه میکنیم: یکی ارتباط حالت فعلی به حالت پیشین و یکی مشاهده فعلی با حالت پنهان فعلی یعنی:

$$P(z_t | z_{t-1}), P(z_t | z_{t-1}).$$

پس در این روش دنیاله حالت ها به صورت مستقیم در دسترس نیستند و دنباله مشاهدات را مستقیماً داریم.

Theoretical Question 14

In the Gaussian-emission HMM used in this project, each hidden state i has parameters μ_i and σ_i^2 . What do these parameters represent?

در روش مارکوف پنهان فرض میشود که مشاهده در هر زمان اگر از حالت پنهان i ام آمده باشد از یک توزیع نرمال گاوسی پیروی کند که میانگین و واریانس این توزیع عمین پارامترها هستند. میانگین مشاهدات همان μ_i که در حالت پنهان i ام دیده میشود یعنی همان مقدار معمول مشاهده شده در این حالت پنهان و واریانس σ_i^2 واریانس مشاهدات در حالت پنهان i است و میزان پراکندگی مشاهدات در این حالت را نشان میدهد که اگر این مقدار کوچک باشد مشاهدات ما نزدیک میانگین هستند و هر چه بزرگتر شود پراکندگی حول میانگین بیشتر خواهد بود. پس در روش مارکوف پنهان میانگین و واریانس که برای توزیع نرمال گاوسی هر روش در نظر میگیریم نوع مشاهداتی که احتمال دارد تولید شود به وسیله این حالت پنهان را توصیف میکند.

Simulation Question 5

Implement a function that generates a sequence from a two-state Gaussian-emission HMM. Your function should take π , A , emission means (μ_1, μ_2) , emission variances (σ_1^2, σ_2^2) , and sequence length T .

It should return both the hidden state sequence z and the observed sequence x .

در این شبیه سازی، یک مولد دنباله بر اساس یک مدل مخفی مارکوف دو حالت با انتشار گاوسی پیاده سازی شد. در این مدل ابتدا مسیر حالت های مخفی بر اساس توزیع اولیه و ماتریس انتقال تولید شده و سپس هر مشاهده با توجه به حالت متناظر از یک توزیع گاوسی نمونه برداری می شود. پارامترهای مورد استفاده در شبیه سازی به صورت زیر انتخاب شدند:

$$\pi = \begin{bmatrix} 0.8 & 0.2 \end{bmatrix}$$

که نشان‌دهنده احتمال اولیه قرارگیری سیستم در هر حالت است. همچنین ماتریس انتقال حالت‌ها برابر است با:

$$A = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}$$

به این معنی که هر حالت با احتمال زیادی در همان حالت باقی می‌ماند و تغییر حالت با احتمال کمتری رخ می‌دهد. برای مدل انتشار گاوسی، پارامترهای میانگین و واریانس دو حالت به صورت زیر در نظر گرفته شدند:

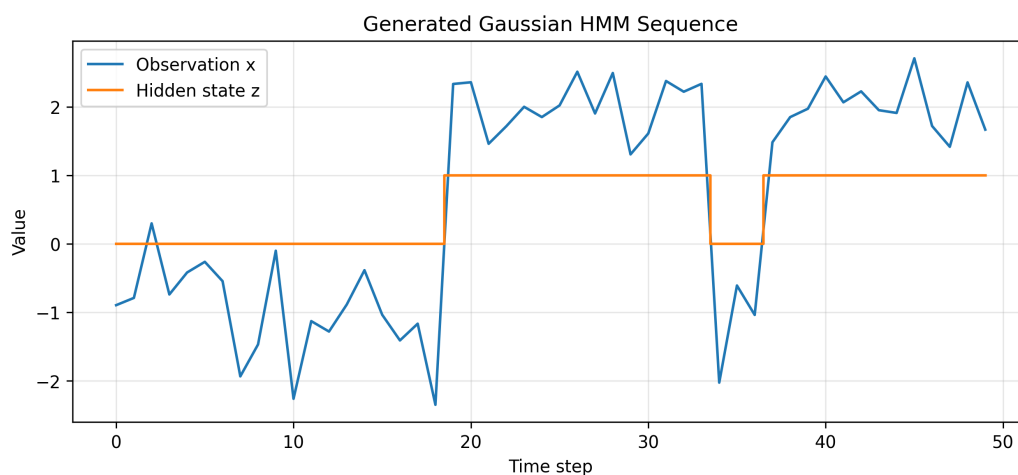
$$\mu = [-1 \quad 2]$$

$$\sigma^2 = [0.6^2 \quad 0.3^2]$$

بنابراین مشاهدات مربوط به حالت اول حول مقدار میانگین -1 و مشاهدات مربوط به حالت دوم حول مقدار میانگین 2 تولید می‌شوند. طول دنباله تولیدشده برابر با:

$$T = 50$$

در نظر گرفته شد. خروجی حاصل از اجرای کد شامل دو بخش است: مسیر واقعی حالت‌های مخفی z و دنباله مشاهدات تولیدشده x . نتیجه شبیه‌سازی در شکل زیر نمایش داده شده است.



شکل ۵: Generated sequence from a two-state Gaussian HMM

همان‌طور که در شکل مشاهده می‌شود، تغییرات دنباله مشاهده‌شده با تغییر حالت‌های مخفی همخوانی دارد. هنگامی که مدل در حالت اول قرار دارد، نمونه‌های تولیدشده در محدوده نزدیک به میانگین -1 قرار گرفته و در حالت دوم، مقادیر حول میانگین 2 هستند این نتیجه نشان می‌دهد که مولد پیاده‌سازی شده توانسته است رابطه بین حالت‌های مخفی و توزیع انتشار گاوسی را به درستی شبیه‌سازی کند.

۴.۴ Most Likely State Sequence and Viterbi Decoding

Theoretical Question 15

Why is brute-force decoding of the most likely hidden state sequence infeasible for long sequences? What is the number of possible paths when there are K states and sequence length T ?

در روش brute-force برای پیدا کردن بهترین مسیر و محتمل ترین دنباله حالت های پنهان باید تمام مسیر های ممکن را در نظر بگیریم و احتمالا هر یک را حساب کرده و سپس دنباله با بیشترین احتمال را انتخاب کنیم حالا اگر طول دنباله برابر (T) باشد، برای زمان اول (K) انتخاب، برای زمان دوم نیز (K) انتخاب و به همین ترتیب برای هر یک از (T) گام زمانی، (K) انتخاب وجود دارد. بنابراین تعداد کل دنباله ها یا مسیرهای ممکن حالت های پنهان برابر است با: K^T در این صورت اگر طول دنباله زیاد شود باید احتمال رخداد کلی مسیر رو حساب کنیم و زمان اجزا بسیار بالا می شود زیرا تعداد مسیر هایی که باید بررسی کنیم به صورت نمایی با طول دنباله رشد میکند و افزایش میابد. پس اردر عملیاتی این کار $O(K^T)$ است در صورتی که در روش الگوریتم ویتروبی اردر عملیاتی برابر $O(TK^2)$ است چون در زمان t برای بررسی امتیاز هر یک از K حالت انتقال از هر یک از K حالت قبلی را محاسبه میکنیم. پس الگوریتم ویتروبی زمان اجرا بسیار کمتر از Brute-Force دارد و عملی تر است (توجه کنید چون الگوریتم ویتروبی روشی بازگشتی است و هر بار لازم نیست بهترین مسیر تا مرحله قبل را محاسبه کنیم و از قبل این مسیر حساب شده و ان را در اختیار داریم)

Simulation Question 6

Implement log-space Viterbi decoding for a two-state Gaussian-emission HMM. Test it on a sequence generated by your HMM generator. Report the fraction of time steps where the decoded hidden state matches the true generated hidden state.

برای مدل انتشار گاوسی، پارامترهای زیر استفاده شدند:

$$\mu = [-0.5 \quad 0.8]$$

و

$$\sigma^2 = [0.8^2 \quad 0.8^2]$$

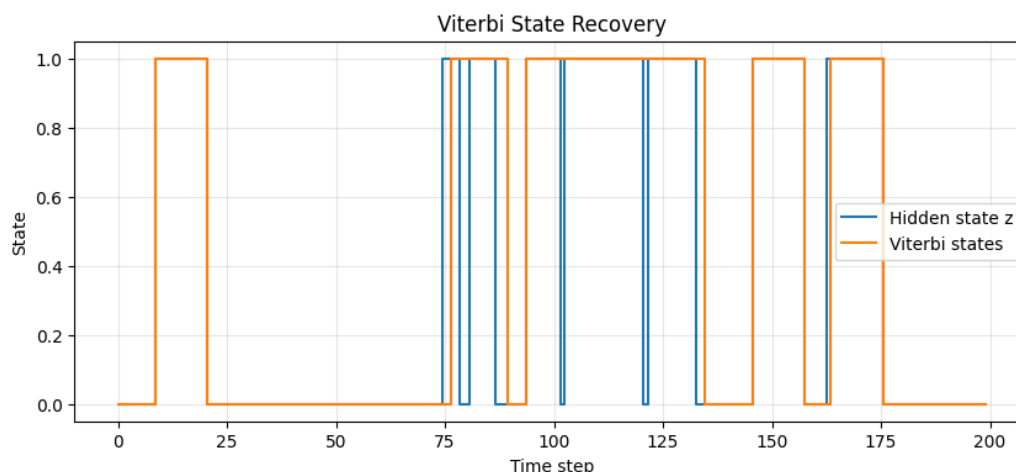
پس از تخمین مسیر حالت ها، خروجی Viterbi با مسیر واقعی تولید شده توسط مولد HMM مقایسه شد. معیار ارزیابی مورد استفاده، درصد بازیابی صحیح حالت ها است که به صورت زیر محاسبه می شود:

$$Accuracy = \frac{1}{T} \sum_{t=1}^T \mathbf{1}(z_t = \hat{z}_t)$$

در این رابطه z_t حالت واقعی تولید شده و $\hat{z}_t$ حالت تخمین زده شده توسط الگوریتم Viterbi است. برای بررسی اثر طول دنباله، الگوریتم روی دو دنباله با طول های مختلف اجرا شد. نتایج حاصل از اجرای کد به صورت زیر به دست آمد:

Length Sequence	Accuracy Recovery State
$T = 50$	$100_{50}\%$
$T = 200$	$94_{200}\%$

در شکل زیر مسیر واقعی حالت های مخفی و مسیر تخمین زده شده توسط Viterbi برای دنباله با طول $T = 200$ مقایسه شده است.



شکل ۶: Comparison between the true hidden state and the state recovered by the Viterbi.

نتایج نشان می‌دهد که الگوریتم Viterbi قادر است با استفاده از مشاهدات تولیدشده، مسیر حالت‌های مخفی مدل را با دقت بالایی بازیابی کند. همچنین افزایش طول دنباله باعث فراهم شدن اطلاعات زمانی بیشتر برای الگوریتم شده و امکان تخمین پایدارتر حالت‌ها را فراهم می‌کند.

۵.۴ Connection to HMAD

Theoretical Question 16

In one or two paragraphs, explain why Viterbi decoding is useful for HMAD. What role does the decoded hidden state sequence play in constructing the feature vector $\Psi(x, z)$?

در روش HMAD فقط دنباله‌ی مشاهدات $x = (x_1, \dots, x_T)$ در اختیار ما قرار دارد و حالت‌های واقعی سیستم $z = (z_1, \dots, z_T)$ پنهان هستند. درحالی‌که برای تشخیص ناهنجاری لازم است علاوه بر مقادیر مشاهدات، بدانیم سیستم در طول زمان احتمالاً در چه حالت‌هایی قرار داشته و چگونه از یک حالت به حالت دیگر منتقل شده است. الگوریتم ویتربی با در نظر گرفتن احتمال انتقال میان حالت‌ها و احتمال تولید هر مشاهده در هر حالت، محتمل‌ترین دنباله‌ی حالت‌های پنهان را پیدا می‌کند:

$$z^* = \arg \max_z P(x, z \mid \lambda).$$

بنابراین، ویتربی برای هر دنباله‌ی مشاهده‌شده یک مسیر پنهان سازگار با کل دنباله پیدا می‌کند. این موضوع مهم است، زیرا طبیعی یا غیرطبیعی بودن یک مشاهده می‌تواند به حالتی که آن را تولید کرده است وابسته باشد؛ برای مثال، یک دمای بالا ممکن است در حالت «بار زیاد» طبیعی باشد، ولی در حالت «عملکرد عادی» مشکوک در نظر گرفته شود. پس از به دست آوردن مسیر z^* ، این دنباله همراه با مشاهدات (x) برای ساخت بردار ویژگی مشترک $\Psi(x, z^*)$ استفاده می‌شود. این بردار فقط اطلاعات مربوط به مقادیر مشاهدات را در خود نگه نمی‌دارد، بلکه ساختار زمانی دنباله را نیز ثبت می‌کند؛ برای مثال، مشخص می‌کند هر انتقال از حالت (i) به حالت (j) چند بار رخ داده و در زمان‌هایی که سیستم در هر حالت بوده، چه مشاهداتی تولید شده‌اند. سپس این نمایش با طول ثابت به یک آشکارساز تک‌کلاسه مانند OC-SVM داده می‌شود تا الگوی معمول دنباله‌های نرمال را یاد بگیرد. در نتیجه، HMAD می‌تواند دنباله‌هایی را که دارای انتقال‌های غیرمعمول یا مشاهدات ناسازگار با حالت‌های پنهان هستند به عنوان ناهنجاری تشخیص دهد؛ حتی اگر هر مشاهده به تنهایی طبیعی به نظر برسد.

۶.۴ Bonus Questions

Theoretical Question 17

A stationary distribution v of a Markov chain satisfies

$$v^T A = v^T, \quad \sum_{i=1}^K v_i = 1.$$

For the transition matrix

$$A = \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix},$$

compute the stationary distribution and interpret it as the long-run fraction of time spent in each state.

توجه کنید چون بردار v پس از انتقال با ماتریس A نیز همچنان در حالت v قرار گرفته است یعنی اگر سیستم به توزیع v روی حالت های خود برسد از این به بعد در همین توزیع میماند و این توزیع همان توزیع مانا آن است که در زمان های طولانی برای سیستم پایدار بدست میاوریم. چون ماتریس A ماتریسی ۲.۲ است پس بردار توزیع اولیه ما باید ۲.۱ باشد یعنی دو حالت در سیستم داریم و بردار توزیع اولیه را میتوان اینگونه نمایش داد:

$$v = (v_1 \ v_2).$$

که در اینجا v_1 همان احتمال حضور در حالت اول و v_2 همان احتمال حضور در حالت دوم است و به تعبیر دیگری میتوان این دو را سهم بلند مدت حضور سیستم در این حالت ها در نظر گرفت.

$$v^T A = v^T.$$

با انجام ضرب ماتریسی داریم دستگاه معادلات زیر به دست می آید:

$$0.95v_1 + 0.10v_2 = v_1, \quad 0.05v_1 + 0.90v_2 = v_2.$$

همچنین: $v_1 + v_2 = 1$.
پس:

$$0.95v_1 + 0.10v_2 = v_1, \rightarrow 0.10v_2 = 0.05v_1, \rightarrow v_1 = 2v_2.$$

با جای گذاری $v_1 = 2v_2$:

$$2v_2 + v_2 = 1, \quad 3v_2 = 1,$$

بنابراین:

$$v_2 = \frac{1}{3}, \quad v_1 = \frac{2}{3}.$$

پس توزیع زنجیره برابر است با:

$$v = \left(\frac{2}{3} \ \frac{1}{3} \right) = (0.6667 \ 0.3333)$$

که این نشان می دهد که سیستم در بلند مدت تقریباً در 66.7 درصد مواقع در حالت اول و در 33.3 درصد مواقع در حالت دوم است. توجه کنید با توجه به ماتریس A این نتیجه منطقی هم هست که احتمال حضور سیستم در حالت اول بیشتر باشد چون سیستم با احتمال بیشتری هنگامی که در حالت اول است در حالت اول میماند و با احتمال بیشتری سیستم از حالت دوم به اول می رود تا از حالت اول به دوم برود.

Theoretical Question 18

The entropy rate of a Markov chain with stationary distribution v and transition matrix A is

$$H = - \sum_{i=1}^K v_i \sum_{j=1}^K A_{ij} \log A_{ij}.$$

Explain qualitatively how a transition matrix with highly predictable transitions differs from a transition matrix with nearly uniform transitions. How might this be useful for anomaly detection?

پس نرخ انروپی میزان عدم قطعیت در انتقال زنجیره مارکوف را بیان میکند و نشان میدهد اگر حالت فعلی را بدانیم حالت بعدی سیستم چه قدر قابل پیش بینی است. یعنی اگر سیستم خیلی قابل پیش بینی باشد در هر سطر ماتریس A که مربوط به انتقال از حالت فعلی که آن را میدانیم به حالت های دیگر است، یکی از درایه ها بسیار به ۱ نزدیک است و باقی درایه ها تقریباً ۰ هستند اینگونه قطعیت بیشتری داریم که از این حالت به کدام حالت میرویم.

در این حالت یا $\log(A_{ij})$ یا خود A_{ij} بسیار کوچک میشود و در نتیجه انتروپی که همان میانگین وزن دار این ها با توجه به توزیع حالت های سیستم یا همان v_i ها است کم شده و در نتیجه کم شدن انتروپی یعنی سیستم قطعیت زیادی برای حالت بعدی در انتقال سیستم از این حالت دارد.

در مقابل اگر توزیع یکتواختی بین حالت های سیستم داشته باشیم یعنی احتمال رفتن از یک حالت به حالت بعدی برابر باشد یا در هر سطر A احتمال ها یک اندازه باشند آنگاه دانستن حالت فعلی زیاد کمکی به پیش بینی حالت بعدی نمیکند چون سیستم با احتمال برابری بین حالت ها جابجا میشود و عدم قطعیت زیاد است و دیگر مقدار $A_{ij} \log(A_{ij})$ به ۰ نزدیک نمیشود و در واقع برای تمامی مقادیر i, j یکسان است و در نتیجه انتروپی بزرگ میشود که نشان دهنده عدم قطعیت زیاد در جایجایی بین حالت هاست.

پس انتروپی را برای ناهنجاری ها میتوان استفاده کرد و کاربرد دارد زیرا اگر سیستم ما قاعده مند باشد و الگوی انتقال های آن یعنی همان ماتریس A را بدانیم که در پروژه ما میدانیم مشاهده حالت ها ناهنجار میان انتقال حالت ها در دنباله رخداد های سیستم باعث افزایش نرخ انتروپی میشود و همین افزایش انتروپی نشان دهنده ناهنجاری و خروج سیستم از حالت نرمال آن است.

البته به طور کلی تغییر ناگهانی نرخ انتروپی میتواند نشان دهنده ناهنجاری در سیستم باشد یعنی ممکن است سیستم در حالت عادی مدام بین حالت های مختلف جابجا شود ولی ناگهان برای مدت طولانی در الگوی ثابت تکراری گیر کند. در این حالت خود این حالت که یعنی کاهش ناگهانی انتروپی میتواند نشان دهنده خرابی در حالت معمول سیستم و ناهنجاری آن باشد.

پس به طور کلی با مشاهده تغییرات نرخ انتروپی میتوان برای تشخیص ناهنجاری ها استفاده کرد. گاهی سیستم در حالت کلی پایدار است و جابجایی زیادی بین حالات مختلف ندارد و با افزایش انتروپی میتوان به ناهنجاری در آن پی برد گاهی نیز سیستم در حالت عادی مدام بین حالات جابجا میشود و کاهش ناگهانی انتروپی که عدم تغییر حالت آن است میتواند نشان دهنده ناهنجاری در سیستم باشد.

Theoretical Question 19

For an HMM, the marginal likelihood of an observation sequence is

$$P(x | \lambda) = \sum_{z \in \mathcal{Z}} P(x, z | \lambda).$$

Explain why computing this sum by brute force is infeasible. Then state the main idea of the Forward algorithm and how it reduces the computational complexity.

در HMM ما فقط دنباله مشاهدات x_i را داریم و از دنباله حالت های پنهان z_i اطلاعی نداریم پس برای محاسبه مارجین در روش Brute-Force باید تمام دنباله های ممکن Z را در نظر بگیریم و احتمال آن ها را حساب کرده و سپس جمع کنیم. این کار از لحاظ زمانی و محاسباتی منطقی نیست چون اگر دنباله ما طول T و تعداد حالت های سیستم نیز K باشد آنگاه باید K^T حالت را محاسبه کنیم که یعنی مقدار محاسبات ما نسبت به طول دنباله حالت های پنهان سیستم در اردر نمایی هستند و هزینه محاسباتی آن زیاد میشود و حتی اگر کمی طول دنباله زیاد شود محاسبه آن غیرممکن میشود. بنابراین روش Forward برای محاسبه مارجین معرفی میشود که به جای در نظر گرفتن کل دنباله در هر مرحله فقط دنباله را تا زمان مورد نظر t در نظر میگیرد و احتمال مسیر هایی که باعث میشود مشاهده $x_1, x_2, \dots, x_i$ رخ دهد و در نهایت در زمان t به حالت j برا سیستم برسد را محاسبه می کند.

تفاوت روش forward با ویتروبی در این است که در روش forward ما این کار را برای تمام مسیر های که به حالت j برسند حساب میکنیم ولی در روش ویتروبی تنها بهترین مسیر را حساب میکنیم که احتمال بیشتری دارد. یعنی روش forward برای محاسبه احتمال کل تمام مسیر های ممکن را جمع میزند ولی ویتروبی فقط محتمل ترین مسیر را نگه میدارد و از بیشینه استفاده میکند. پس روش forward احتمال کل دنباله های حالت های پنهان را با جمع تمام مسیر های ممکن حساب میکند ولی ویتروبی محتمل ترین دنباله را با انتخاب بهترین مسیر پیدا میکند. روش کار forward در مرحله اولیه داریم:

$$\pi_j P(x_1 | z_1 = j).$$

یعنی احتمال شروع از حالت (j) در احتمال تولید مشاهده ی اول در همان حالت ضرب می شود. سپس رابطه ی بازگشتی Forward به صورت زیر است:

$$P(x_{t+1} | z_{t+1} = j) \sum_{i=1}^K \alpha_t(i) A_{ij}.$$

برای محاسبه ی $\alpha_{t+1}(j)$ ، ابتدا تمام راه هایی را که می توانند از حالت های قبلی (i) به حالت فعلی (j) برسند در نظر می گیریم. $\sum_{i=1}^K \alpha_t(i) A_{ij}$ سپس این مقدار را در احتمال تولید مشاهده ی جدید x_{t+1} در حالت (j) ضرب می کنیم:

$$P(x_{t+1} | z_{t+1} = j).$$

در پایان، احتمال حاشیه ای کل دنباله ی مشاهدات با جمع کردن احتمال رسیدن به تمام حالت های ممکن در زمان (T) محاسبه می شود:

$$\sum_{j=1}^K \alpha_T(j).$$

در هر گام زمانی، برای محاسبه ی مقدار Forward هر یک از (K) حالت فعلی، باید انتقال از (K) حالت قبلی بررسی شود. بنابراین در هر زمان تقریباً K^2 محاسبه انجام می شود و برای (T) گام زمانی، پیچیدگی الگوریتم برابر است با: $O(TK^2)$. که پیچیدگی زمانی کمتری دارد پس عملی تر است چون بازگشتی عمل کردیم و در هر مرحله مقادیر را ذخیره کردیم و اینگونه از محاسبه مقدار های تکراری جلوگیری کردیم.

Theoretical Question 20

Posterior decoding chooses the most likely state at each individual time step:

$$\hat{z}_t = \arg \max_i P(z_t = i | x, \lambda).$$

Viterbi decoding chooses the globally most likely state sequence. Explain why posterior decoding can sometimes produce an invalid or unlikely global path, while Viterbi respects the transition structure of the model.

در رمزگشایی پسین، برای هر زمان t ، احتمال حضور در هر حالت با توجه به کل دنباله‌ی مشاهدات محاسبه می‌شود و سپس حالتی که بیشترین احتمال پسین را دارد انتخاب می‌شود:

$$\arg \max_i P(z_t = i \mid x, \lambda).$$

مشکل این روش آن است که انتخاب حالت در هر زمان، به صورت جداگانه انجام می‌شود. یعنی هنگام انتخاب $\hat{z}_t$ ، الزاماً بررسی نمی‌شود که انتقال از حالت انتخاب‌شده در زمان $(t-1)$ به حالت انتخاب‌شده در زمان t چقدر احتمال دارد. در نتیجه، مجموعه‌ی حالت‌هایی که در زمان‌های مختلف به صورت مستقل بیشترین احتمال را داشته‌اند ممکن است در کنار هم یک مسیر کم‌احتمال یا حتی نامعتبر ایجاد کنند. یعنی این الگوریتم دنباله حالت‌های پنهان تا زمان t را ایگنور کرده و مستقلاً محتمل‌ترین حالت را برای زمان t در نظر می‌گیرد هر چند ممکن است احتمال جابجایی بین حالت z_{t-1} ، z_t کم باشد. بنابراین ممکن است در آخر الگوریتم به دنباله‌ای از حالت‌های پنهان برسیم که درست است که احتمال هر درایه آن مستقلاً ماکس است ولی کل دنباله احتمال کمی داشته باشد.

در مقابل، الگوریتم ویتربی حالت‌ها را به صورت جداگانه انتخاب نمی‌کند، بلکه کل دنباله‌ی حالت‌ها را هم‌زمان در نظر می‌گیرد:

$$\arg \max_z P(x, z \mid \lambda).$$

احتمال مشترک یک مسیر در HMM شامل هم احتمال تولید مشاهدات و هم احتمال انتقال میان حالت‌هاست:

$$P(z_1)P(x_1 \mid z_1) \prod_{t=2}^T P(z_t \mid z_{t-1})P(x_t \mid z_t).$$

بنابراین، ویتربی هنگام انتخاب مسیر بررسی می‌کند که هر حالت تا چه اندازه با مشاهده‌ی مربوط به خود سازگار است و همچنین انتقال میان دو حالت متوالی چقدر محتمل است. به همین دلیل ممکن است ویتربی در یک زمان، حالتی را انتخاب کند که به تنهایی بیشترین احتمال پسین را ندارد، اما قرارگرفتن آن حالت در کنار حالت‌های قبل و بعد، مسیر کلی محتمل‌تری می‌سازد. پس ویتربی دنباله‌ای را انتخاب می‌کند که احتمال مشترک آن زیاد باشد و اینگونه ساختار زمانی را حفظ می‌کند و وابستگی‌های زمانی را در نظر می‌گیرد.

Simulation Question 7

Compare ordinary probability multiplication and log-space computation for a long Markov chain sequence.

Generate a sequence of length $T = 5000$, compute its probability both as a raw product and as a log-probability sum, and show when the raw product begins to suffer from numerical underflow.

در این شبیه‌سازی، هدف بررسی مشکل محاسبه احتمال مسیرهای طولانی در زنجیره مارکوف و مقایسه روش ضرب مستقیم احتمال‌ها با روش محاسبه در فضای لگاریتمی است. برای این کار، یک مسیر حالت مارکوف با طول

$$T = 5000$$

تولید شد. سپس احتمال این مسیر به دو روش محاسبه گردید. در روش اول، احتمال مسیر با ضرب مستقیم احتمال‌های انتقال محاسبه شد:

$$P(z \mid \pi, A) = \pi_{z_1} \prod_{t=2}^T A_{z_{t-1}, z_t}$$

در روش دوم، به جای ضرب احتمال‌های کوچک، مجموع لگاریتم احتمال‌ها محاسبه شد:

$$\log P(z \mid \pi, A) = \log \pi_{z_1} + \sum_{t=2}^T \log A_{z_{t-1}, z_t}$$

نتایج حاصل از اجرای کد به صورت زیر به دست آمد:

Length: Sequence $T = 5000$
 probability: Direct 0.0
 probability: Log - 1343.6271123889937
 log: from Probability 0.0

مشاهده می‌شود که در روش ضرب مستقیم، به دلیل ضرب تعداد بسیار زیادی احتمال کوچک، مقدار نهایی دچار مشکل *numerical underflow* شده و مقدار صفر گزارش می‌شود. در مقابل، روش فضای لگاریتمی با تبدیل ضرب‌های متوالی به جمع، از کاهش شدید دقت جلوگیری کرده و مقدار لگاریتم احتمال را با دقت مناسب محاسبه می‌کند. بنابراین، استفاده از محاسبات لگاریتمی برای مدل‌های دنباله‌ای طولانی ضروری است.

۵ Hidden Markov Anomaly Detection

۱.۵ Conceptual Bridge from OC-SVM to HMAD

Theoretical Question 21

OC-SVM uses a fixed feature map $\phi(x)$, while HMAD uses a joint feature map $\Psi(x, z)$ with a maximization over hidden states.

Explain in your own words what this maximization is doing. Why can it help identify anomalies that a fixed feature map would miss? Use an example to make your case.

در OC-SVM ما بردار ویژگی‌های ورودی x را به فضای ویژگی‌های $\phi(x)$ نگاشت می‌کنیم که به بردار ثابت ویژگی‌ها تبدیل شوند و بر اساس این تبدیل امتیاز نرمال بودن را محاسبه می‌کنیم. در این روش اگر اطلاعات میان ترتیب زمانی و وابستگی‌های ویژگی‌ها در دنباله $\phi(x)$ ثبت نشده باشد دیگر مدل نمی‌تواند اطلاعات مربوط به ترتیب زمانی این اطلاعات را اثر دهد و مثلاً در نظر بگیرید که بردار ویژگی‌هایی که ما می‌سازیم شامل میانگین و واریانس و کمینه و ماکس اطلاعات بردار x باشد در این صورت ترتیب اطلاعات در بردار x اثری ندارد و در $\phi(x)$ اثری نمی‌گذارد و هر ترتیب مختلف از اطلاعات در x که به همین اطلاعات برسند از نظر مدل یکسان هستند در نتیجه مدل اصلاً به دنباله اطلاعات و وابستگی ترتیب زمانی آنها اهمیتی نمی‌دهد.

در مقابل در HMAD نگاشت ویژگی‌ها به صورت نگاشت ویژگی مشترک $\Psi(x, z)$ است که علاوه بر مشاهدات x به دنباله حالت‌های پنهان z نیز توجه می‌کند و سپس تابع امتیازدهی را بر اساس تمامی دنباله‌های حالت‌های پنهان انجام می‌دهد. خود در نظر گرفتن دنباله حالت‌های پنهان یعنی در نظر گرفتن اثر ترتیب زمانی هر حالت و سپس امتیازدهی به وسیله ماکس گرفتن بین تمام حالت‌های دنباله z انجام می‌شود و مسیری را انتخاب می‌کند که بیشترین سازگاری با الگوی نرمالی که مدل یاد گرفته داشته باشد زیرا اینکه ماکس امتیاز را در نظر می‌گیرد و هر چه امتیاز بیشتر باشد یعنی داده نرمال تر بوده یعنی با احتمال و قطعیت بیشتری نرمال است یعنی همان که به وسیله این ماکس‌گیری از دنباله‌ای از حالت‌های پنهان را در نظر بگیرد که بیشترین سازگاری با الگوی نرمال ترین شده داشته باشد.

یعنی به دنبال آن دنباله‌ای از حالت‌ها می‌گردد که هم انتقال میان آنها طبیعی باشد و مشاهدات تولید شده با الگوی نرمال سازگار باشند. سپس امتیاز را با این دنباله محاسبه می‌کند که مقداری بزرگی است و در نتیجه امتیاز نرمال بودن بالایی دریافت کند ولی اگر هیچ دنباله‌ای از حالت‌های پنهان که برای این مشاهدات در الگوی نرمال صدق کنند وجود نداشته باشد آنگاه امتیاز نرمال بودن پایین خواهد بود و به عنوان ناهنجاری گزارش می‌شود.

پس مزیت بیشینه‌گیری روی حالت‌های پنهان این است که HMAD برای هر دنباله بهترین توضیح ساختاری ممکن را جست‌وجو می‌کند. اگر حتی بهترین توضیح نیز با الگوی نرمال سازگار نباشد، دنباله ناهنجار در نظر گرفته می‌شود. به همین دلیل HMAD می‌تواند ناهنجاری‌های مربوط به ترتیب زمانی، انتقال میان حالت‌ها و رابطه‌ی مشاهدات با حالت‌های

پنهان را تشخیص دهد؛ درحالی که یک نگاشت ویژگی ثابت ممکن است این اطلاعات را در فرایند خلاصه سازی از دست بدهد.

برای مثال دو دنباله ای از مشاهدات را در نظر بگیرید که شامل ۱۰ تا دمای ۲۰ و ۱۰ تا دمای ۳۰ باشد. در دنباله اول ابتدا دمای ۲۰ مشاهده شده و سپس دمای ۳۰ و در دنباله دوم یکی در میان دمای ۲۰ و ۳۰ مشاهده شود. در این دو دنباله ماکس و مین و میانگین و واریانس مشاهدات برابر هستند ولی ترتیب زمانی است که تفاوت را ایجاد میکند زیرا برای یک سیستم انتقال های پی در پی دمایی و تغییرات دمایی زیاد ممکن است ناسازگار باشد. برای این دو دنباله در روش اول بردار ویژگی های $\phi(x)$ هیچ تفاوتی میان دو دنباله نمی بیند و مدل هر دو را یک جور ارزیابی میکند ولی برای روش دوم چون در دنباله حالت های پنهان z برای مثال برای دمای ۲۰ در حالت استراحت و حالت ۱ و برای دمای ۳۰ در حالت پرکار و حالت ۲ در نظر میگرد پس دنباله های حالت پنهان برای این دو متفاوت هستند و چون تغییر مداوم بین حالت های ۱، ۲ برای این سیستم ناهنجاری است مدل ما بین این دو مشاهده تفاوت می بیند و یکی را به عنوان ناهنجاری با امتیاز کم و دیگری را الگوی نرمال با امتیاز بالا در نظر میگیرد و بدین ترتیب اثر ترتیب زمانی در دنباله مشاهدات را با دنباله های پنهان در نظر میگیرد.

Theoretical Question 22

Suppose two sequences $x^{(1)}$ and $x^{(2)}$ have identical summary statistics, including the same mean, variance, minimum, and maximum.

Can HMAD still assign them different anomaly scores? Explain why or why not.

مطابق توضیحات قسمت قبل بله، HMAD می تواند حتی در صورتی که دو دنباله دارای میانگین، واریانس، کمینه و بیشینه ی یکسان باشند، به آن ها امتیازهای ناهنجاری متفاوتی اختصاص دهد. دلیل این موضوع آن است که آمارهای خلاصه فقط اطلاعات کلی درباره ی مقادیر موجود در دنباله را ثبت می کنند و معمولاً ترتیب زمانی، الگوی انتقال میان حالت ها و نحوه ی توزیع مشاهدات در طول زمان را نشان نمی دهند. برای مثال، دو دنباله ی زیر را در نظر می گیریم:

$$x^{(1)} = (0, 0, 0, 0, 4, 4, 4, 4), x^{(2)} = (0, 4, 0, 4, 0, 4, 0, 4).$$

این دو دنباله دقیقاً شامل چهار مقدار صفر و چهار مقدار چهار هستند. بنابراین میانگین، واریانس، کمینه و بیشینه ی آن ها یکسان است پس اگر فقط از یک بردار ویژگی خلاصه مانند

$$\phi(x) = (\text{mean}, \text{variance}, \text{minimum}, \text{maximum})$$

استفاده شود، نمایش ویژگی این دو دنباله یکسان خواهد بود:

$$\phi(x^{(1)}) = \phi(x^{(2)}).$$

در نتیجه، یک آشکارساز مانند OC-SVM که فقط همین بردار ویژگی را دریافت می کند، نمی تواند تفاوت ترتیب زمانی این دو دنباله را مشاهده کند و احتمالاً امتیاز یکسان یا بسیار مشابهی به آن ها می دهد. اما HMAD از نگاشت ویژگی مشترک

$$\Psi(x, z)$$

استفاده می کند که علاوه بر مقادیر مشاهده شده، دنباله ی حالت های پنهان (z) را نیز در نظر می گیرد. فرض کنیم مقادیر نزدیک صفر به حالت پنهان اول و مقادیر نزدیک چهار به حالت پنهان دوم مربوط باشند. در این صورت، مسیر حالت های پنهان دنباله ی اول تقریباً به شکل زیر است:

$$z^{(1)} = (1, 1, 1, 1, 2, 2, 2, 2).$$

این مسیر فقط یک انتقال بین دو حالت دارد اما مسیر حالت های پنهان دنباله ی دوم تقریباً برابر است با:

$$z^{(2)} = (1, 2, 1, 2, 1, 2, 1, 2),$$

که دارای تعداد زیادی انتقال رفت و برگشتی است بنابراین، اگرچه آمارهای خلاصه‌ی دو دنباله یکسان هستند، تعداد و نوع انتقال‌های حالت در آن‌ها کاملاً متفاوت است. نگاشت مشترک $\Psi(x, z)$ می‌تواند این تفاوت را از طریق ویژگی‌هایی مانند تعداد انتقال‌ها ثبت کند برای دنباله‌ی اول، تعداد انتقال‌های ماندن در یک حالت زیاد و تعداد انتقال میان دو حالت کم است. اما برای دنباله‌ی دوم، تعداد انتقال‌های و بسیار بیشتر خواهد بود. بنابراین داریم:

$$\Psi(x^{(1)}, z^{(1)}) \neq \Psi(x^{(2)}, z^{(2)}).$$

تابع امتیازدهی HMAD نیز بر اساس این نمایش ساختاری محاسبه می‌شود:

$$f(x) = \max_{z \in \mathcal{Z}} \langle w, \Psi(x, z) \rangle - \rho.$$

اگر مدل روی دنباله‌های نرمالی آموزش دیده باشد که معمولاً شامل بلوک‌های پایدار و تعداد کمی انتقال میان حالت‌ها هستند، دنباله‌ی اول با این الگوی نرمال سازگارتر خواهد بود و امتیاز نرمال بودن بیشتری دریافت می‌کند. در مقابل، دنباله‌ی دوم به دلیل جابه‌جایی‌های مکرر میان حالت‌ها ممکن است امتیاز پایین‌تری بگیرد و ناهنجار تشخیص داده شود. البته متفاوت شدن امتیازها به این شرط است که نگاشت $\Psi(x, z)$ واقعاً اطلاعات مربوط به ساختار زمانی و انتقال‌های حالت را در خود ثبت کند. HMAD با بررسی مسیر حالت‌های پنهان، تعداد و نوع انتقال‌ها و رابطه‌ی مشاهدات با هر حالت، می‌تواند تفاوت ساختاری میان دو دنباله را تشخیص دهد و به آن‌ها امتیازهای ناهنجاری متفاوتی اختصاص دهد.

Simulation Question 8

To build intuition for why the joint feature map captures structure that summary statistics miss, construct the following toy experiment in Python.

- Generate two sequences of length $T = 100$, both consisting of the same 50 values drawn from $\mathcal{N}(0, 1)$ and 50 values drawn from $\mathcal{N}(4, 1)$, but arranged differently. In $x^{(1)}$, the two groups form two contiguous blocks: the first 50 steps are drawn from $\mathcal{N}(0, 1)$ and the last 50 steps are drawn from $\mathcal{N}(4, 1)$. In $x^{(2)}$, the values are randomly shuffled.
- Verify that the two sequences have nearly identical global summary statistics: mean, variance, minimum, and maximum. Plot both sequences side by side.
- Manually assign hidden states using the threshold rule

$$z_t = \begin{cases} 1, & x_t < 2, \\ 2, & x_t \geq 2. \end{cases}$$

Compute the joint feature vector $\Psi(x, z)$ for both sequences using $K = 2$ and $\phi(x_t) = x_t$.

- Compare the joint feature vectors of the two sequences. Which entries differ the most, and why?

در این شبیه‌سازی هدف بررسی این موضوع است که چگونه ویژگی مشترک $\Psi(x, z)$ می‌تواند اطلاعات ساختاری دنباله را استخراج کند؛ اطلاعاتی که توسط آمارهای خلاصه مانند میانگین، واریانس، کمینه و بیشینه قابل مشاهده نیستند.

a) در این بخش دو دنباله با طول

$$T = 100$$

تولید شدند. هر دو دنباله شامل ۵۰ نمونه از توزیع

$$\mathcal{N}(0, 1)$$

و ۵۰ نمونه از توزیع

$$\mathcal{N}(4, 1)$$

هستند. تفاوت دو دنباله تنها در ترتیب قرارگیری نمونه‌ها است. در دنباله اول، دو گروه به صورت دو بلوک متوالی قرار گرفته‌اند، در حالی که در دنباله دوم همان نمونه‌ها به صورت تصادفی درهم‌ریزی شده‌اند.

$$x^{(1)} = [\underbrace{\mathcal{N}(0, 1), \dots, \mathcal{N}(0, 1)}_{50}, \underbrace{\mathcal{N}(4, 1), \dots, \mathcal{N}(4, 1)}_{50}]$$

و

$$x^{(2)} = \text{Shuffle}(x^{(1)})$$

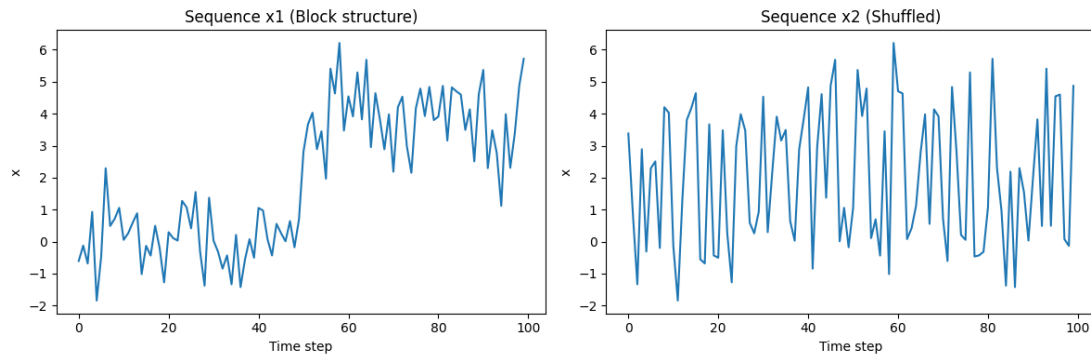
در نتیجه، هر دو دنباله شامل دقیقاً یک مجموعه نمونه هستند، اما ساختار زمانی متفاوتی دارند.

b) برای بررسی اینکه آیا ویژگی‌های آماری ساده قادر به تشخیص تفاوت دو دنباله هستند، مقادیر میانگین، واریانس، کمینه و بیشینه محاسبه شدند.

جدول ۱۰: Global summary statistics of two sequences

Statistic	$x^{(1)}$	$x^{(2)}$
Mean	1.977210	1.977210
Variance	4.540181	4.540181
Minimum	-1.844401	-1.844401
Maximum	6.212747	6.212747

همان‌طور که مشاهده می‌شود، این دو دنباله دارای آمارهای کلی تقریباً یکسان هستند، زیرا نمونه‌های تشکیل‌دهنده آن‌ها یکسان است. بنابراین ویژگی‌های خلاصه قادر به ثبت تفاوت موجود در ترتیب زمانی داده‌ها نیستند. شکل زیر دو دنباله تولیدشده را مقایسه می‌کند.



شکل ۷: Comparison of shuffled and block-structured sequences.

c) در این بخش، حالت‌های مخفی به صورت دستی با استفاده از قانون آستانه‌ای زیر تعیین شدند:

$$z_t = \begin{cases} 1, & x_t < 2 \\ 2, & x_t \geq 2 \end{cases}$$

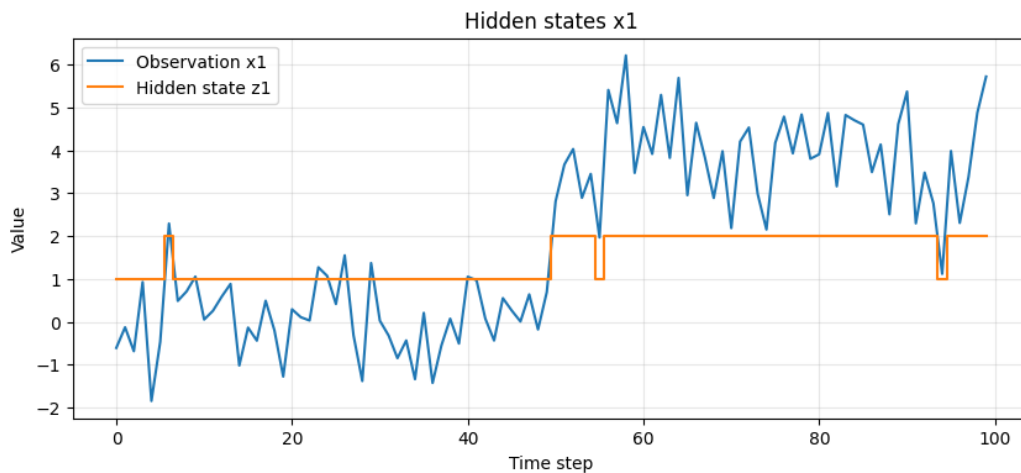
سپس ویژگی مشترک

$$\Psi(x, z)$$

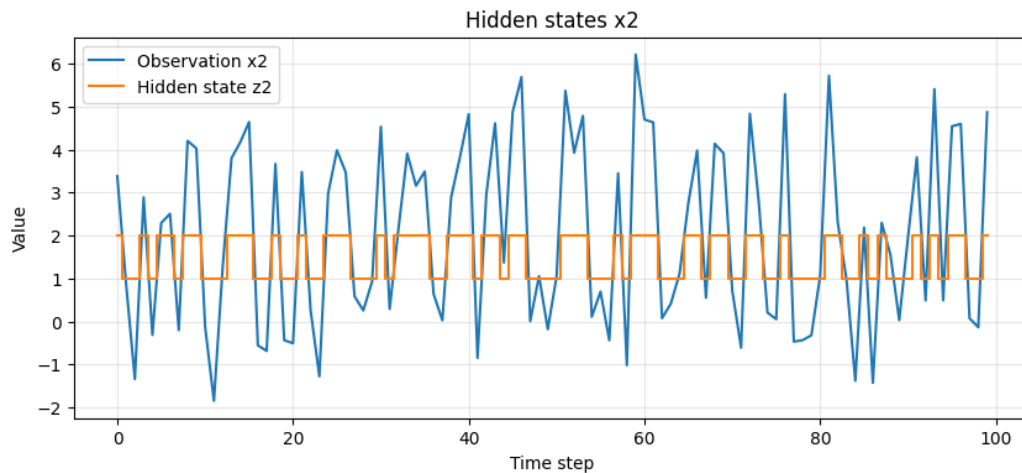
برای هر دو دنباله محاسبه شد. در این آزمایش، بردار ویژگی شامل ویژگی‌های انتشار و انتقال بین حالت‌ها است:

$$\Psi(x, z) = [e_1, e_2, c_{11}, c_{12}, c_{21}, c_{22}]$$

که در آن e_i مجموع مقادیر مشاهده‌شده در حالت i و c_{ij} تعداد انتقال از حالت i به حالت j را نشان می‌دهد. نتایج تخصیص حالت‌های مخفی برای دو دنباله در شکل‌های زیر نمایش داده شده است.



شکل ۸: Hidden states assigned to sequence $x^{(1)}$.



شکل ۹: $x^{(2)}$ sequence to assigned states Hidden

d) در این بخش، بردارهای ویژگی مشترک دو دنباله با یکدیگر مقایسه شدند:

$$\Psi(x^{(1)}, z^{(1)})$$

و

$$\Psi(x^{(2)}, z^{(2)})$$

نتایج مقایسه در جدول زیر آورده شده است.

جدول ۱۱: Comparison of joint feature vectors

Feature	$\Psi(x^{(1)}, z^{(1)})$	$\Psi(x^{(2)}, z^{(2)})$	Difference
Emission - State ۱	4.727	4.727	0.000
Emission - State ۲	192.994	192.994	0.000
Transition 1 → 1	47.000	25.000	22.000
Transition 1 → 2	4.000	26.000	22.000
Transition 2 → 1	3.000	26.000	23.000
Transition 2 → 2	45.000	22.000	23.000

بیشترین اختلاف مربوط به ویژگی‌های انتقالی است. در دنباله اول به دلیل وجود دو بلوک مجزا، تغییر حالت‌ها محدود بوده و تعداد انتقال‌های بین حالت‌ها کم است. در مقابل، در دنباله دوم به دلیل درهم‌ریختگی نمونه‌ها، تغییرات بیشتری بین حالت‌ها رخ می‌دهد و مقادیر ویژگی‌های انتقالی افزایش پیدا می‌کنند.

بنابراین، اگرچه دو دنباله دارای آمارهای کلی مشابه هستند، ویژگی مشترک $\Psi(x, z)$ می‌تواند تفاوت ساختار زمانی آن‌ها را ثبت کند. این موضوع نشان می‌دهد که استفاده از اطلاعات ترتیبی در مدل‌های دنباله‌ای مانند HMAD باعث استخراج ویژگی‌هایی می‌شود که با روش‌های آماری ساده قابل دستیابی نیستند.

۲.۵ Hidden Markov Joint Feature Map

Theoretical Question 23

For $K = 2$ hidden states and scalar observations ($d = 1$), write out all entries of $\Psi(x, z)$ explicitly.

How many entries does the feature vector have? What does each entry represent?

در نگاشت ویژگی مشترک HMAD، دو نوع ویژگی داریم: ویژگی‌های انتقال که نحوه‌ی جابه‌جایی میان حالت‌های پنهان را ثبت می‌کنند و ویژگی‌های انتشار که مقادیر مشاهده‌شده در هر حالت پنهان را خلاصه می‌کنند. فرم کلی بردار ویژگی برابر است با:

$$[(c_{ij}(z))_{i,j \in \mathcal{V}}; (e_i(x, z))_{i \in \mathcal{V}}].$$

برای دو حالت پنهان داریم: $\mathcal{V} = 1, 2$ بنابراین چهار انتقال ممکن وجود دارد:

$$1 \rightarrow 1, \quad 1 \rightarrow 2, \quad 2 \rightarrow 1, \quad 2 \rightarrow 2.$$

ویژگی انتقال $c_{ij}(z)$ تعداد دفعاتی را نشان می‌دهد که مسیر پنهان از حالت (i) به حالت (j) منتقل شده است:

$$\sum_{t=2}^T \mathbf{1}_{z_{t-1} = i, z_t = j}.$$

در نتیجه، چهار ویژگی انتقال به‌صورت زیر هستند:

$$\sum_{t=2}^T \mathbf{1}_{z_{t-1} = 1, z_t = 1},$$

که تعداد دفعات باقی‌ماندن سیستم در حالت اول را نشان می‌دهد؛

$$\sum_{t=2}^T \mathbf{1}_{z_{t-1} = 1, z_t = 2},$$

که تعداد انتقال‌ها از حالت اول به حالت دوم را نشان می‌دهد؛

$$\sum_{t=2}^T \mathbf{1}_{z_{t-1} = 2, z_t = 1},$$

که تعداد انتقال‌ها از حالت دوم به حالت اول را نشان می‌دهد؛

$$\sum_{t=2}^T \mathbf{1}_{z_{t-1} = 2, z_t = 2},$$

که تعداد دفعات باقی‌ماندن سیستم در حالت دوم را نشان می‌دهد.
چون مشاهدات اسکالر هستند، داریم:

$$d = 1, \quad \phi(x_t) = x_t.$$

ویژگی انتشار مربوط به حالت (i) برابر است با:

$$\sum_{t=1}^T \mathbf{1}(z_t = i)(x_t).$$

پس برای دو حالت داریم:

$$\sum_{t=1}^T \mathbf{1}(z_t = 1)(x_t),$$

که مجموع مقادیر مشاهده شده در زمان هایی است که سیستم در حالت پنهان اول قرار داشته است، و

$$\sum_{t=1}^T \mathbf{1}(z_t = 2)(x_t),$$

که مجموع مقادیر مشاهده شده در زمان هایی است که سیستم در حالت پنهان دوم قرار داشته است. بنابراین بردار ویژگی مشترک به طور کامل برابر است با:

$$[c_{11}(z) \ c_{12}(z) \ c_{21}(z) \ c_{22}(z) \ e_1(x, z) \ e_2(x, z)]$$

تعداد کل مؤلفه ها از رابطه ی زیر به دست می آید:

$$K^2 + Kd. = 2^2 + 2(1) = 4 + 2 = 6.$$

پس:

$$\Psi(x, z) \in \mathbb{R}^6.$$

مؤلفه	معنا
c_{11}	تعداد انتقال های حالت ۱ به حالت ۱
c_{12}	تعداد انتقال های حالت ۱ به حالت ۲
c_{21}	تعداد انتقال های حالت ۲ به حالت ۱
c_{22}	تعداد انتقال های حالت ۲ به حالت ۲
e_1	مجموع مشاهدات مربوط به حالت ۱
e_2	مجموع مشاهدات مربوط به حالت ۲

چهار مؤلفه ی اول ساختار زمانی و الگوی انتقال میان حالت ها را ثبت می کنند و دو مؤلفه ی آخر نشان می دهند که در هر حالت پنهان چه مقادیری مشاهده شده است.

Simulation Question 9

Write a Python function

`compute_joint_features(x,z,K)`

that computes $\Psi(x, z)$ for a scalar sequence x and a hard state assignment z , using $\phi(x_t) = x_t$.

(a) Your function should return a single NumPy vector of length $K^2 + K$.

The first K^2 entries should be the normalized transition counts

$$\frac{c_{ij}(z)}{T-1},$$

so that they sum to approximately one.

The last K entries should be the per-state mean emissions

$$\frac{e_i(x, z)}{\text{state of count}_i + \epsilon}.$$

- (b) Generate 200 nominal sequences of length $T = 100$ from $\mathcal{N}(0, 1)$ and 50 anomalous sequences by inserting a block of 20 steps from $\mathcal{N}(4, 1)$ at a random position.

Use the threshold rule

$$z_t = \begin{cases} 1, & x_t < 2, \\ 2, & x_t \geq 2 \end{cases}$$

to assign states.

- (c) Compute $\Psi(x_i, z_i)$ for every sequence. Plot the resulting 250×6 feature matrix as a heatmap, with normal and anomalous sequences grouped.

Which entries of Ψ visually separate the two groups?

در این شبیه‌سازی هدف بررسی توانایی ویژگی مشترک $\Psi(x, z)$ در نمایش تفاوت بین دنباله‌های نرمال و ناهنجار است. برای هر دنباله ابتدا حالت‌های مخفی با استفاده از یک قانون آستانه‌ای تعیین شده و سپس ویژگی‌های انتقالی و انتشار محاسبه شدند.

a) در این بخش تابعی برای محاسبه بردار ویژگی مشترک

$$\Psi(x, z)$$

پیاده‌سازی شد. مطابق صورت سوال، برای یک مدل با

$$K = 2$$

طول بردار ویژگی برابر است با:

$$K^2 + K = 6$$

چهار مولفه اول مربوط به احتمال‌های انتقال بین حالت‌ها هستند:

$$\frac{c_{ij}}{T - 1}$$

که در آن c_{ij} تعداد انتقال از حالت i به حالت j است. دو مولفه آخر مربوط به میانگین انتشار هر حالت هستند بنابراین بردار نهایی به صورت زیر ساخته می‌شود:

$$\Psi(x, z) = [c_{11}, c_{12}, c_{21}, c_{22}, e_1, e_2]$$

b) در این بخش یک مجموعه داده شامل ۲۵۰ دنباله تولید شد. ۲۰۰ دنباله به عنوان نمونه نرمال از توزیع

$$\mathcal{N}(0, 1)$$

با طول

$$T = 100$$

تولید شدند.

همچنین ۵۰ دنباله ناهنجار ساخته شدند. برای ایجاد ناهنجاری، یک بلوک ۲۰ نمونه‌ای از توزیع

$$\mathcal{N}(4, 1)$$

در یک موقعیت تصادفی از دنباله قرار داده شد.

پس از تولید داده‌ها، حالت‌های مخفی با قانون زیر تعیین شدند:

$$z_t = \begin{cases} 1, & x_t < 2 \\ 2, & x_t \geq 2 \end{cases}$$

سپس برای هر دنباله، بردار ویژگی مشترک محاسبه شد.

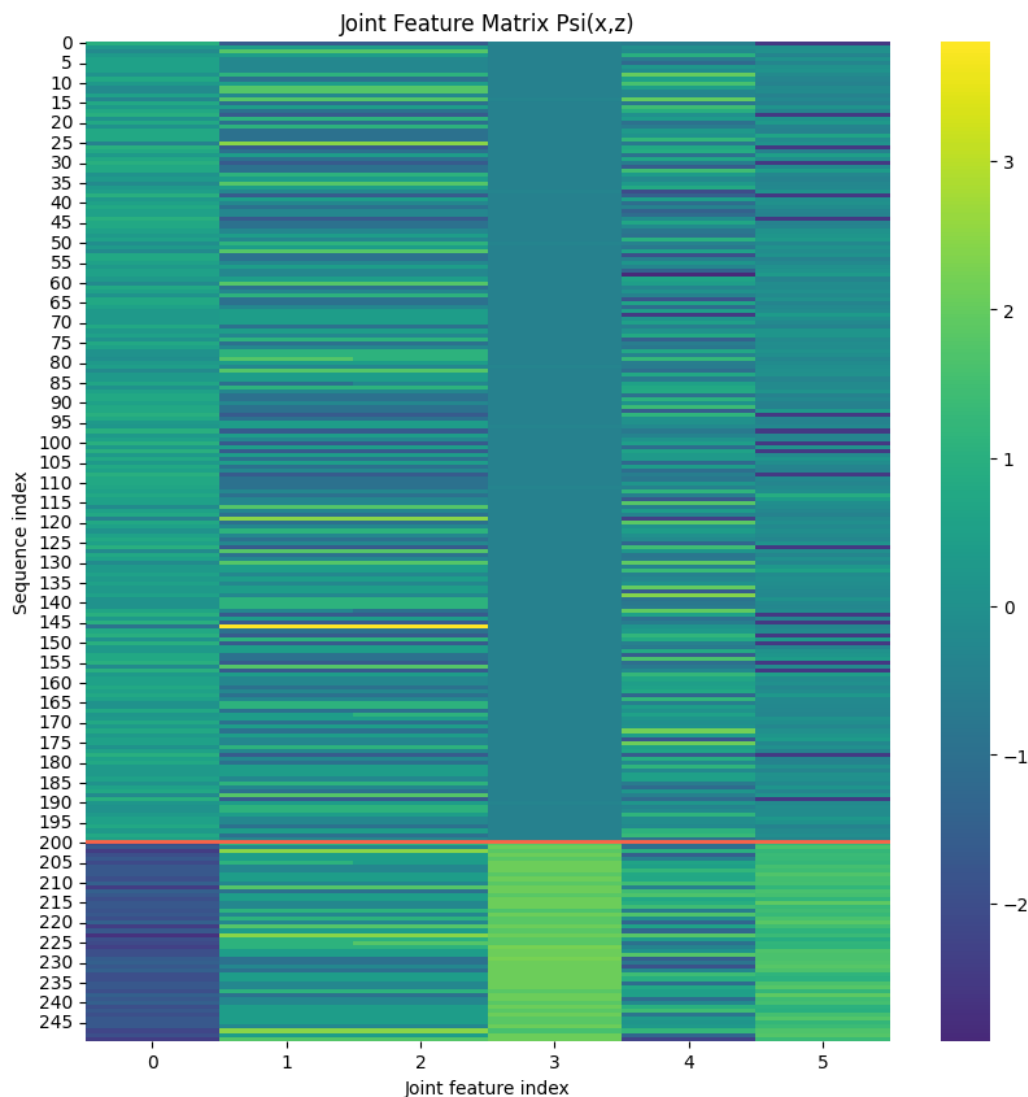
c) در نهایت ویژگی‌های استخراج شده از تمام دنباله‌ها در یک ماتریس قرار گرفتند:

$$\Psi = \begin{bmatrix} \Psi_1 \\ \Psi_2 \\ \vdots \\ \Psi_{250} \end{bmatrix}$$

که ابعاد آن برابر است با:

$$250 \times 6$$

برای مشاهده تفاوت میان نمونه‌های نرمال و ناهنجار، ماتریس ویژگی پس از استانداردسازی به صورت نقشه حرارتی نمایش داده شد.



شکل ۱۰: $\Psi(x, z)$ Heatmap of the joint feature matrix

با توجه به شکل، نمونه‌های نرمال و ناهنجار در بخش‌هایی از فضای ویژگی تفاوت نشان می‌دهند. این تفاوت بیشتر در ویژگی‌های مربوط به حالت دوم و ویژگی‌های انتقالی مشاهده می‌شود، زیرا وجود بلوک ناهنجار باعث افزایش حضور حالت مربوط به مقادیر بزرگ‌تر و تغییر الگوی انتقال بین حالت‌ها می‌شود. این نتیجه نشان می‌دهد که ویژگی مشترک $\Psi(x, z)$ قادر است اطلاعات ساختاری و زمانی دنباله را استخراج کند و تنها به آمار کلی مقادیر وابسته نیست.

۳.۵ Simplified HMAD Training Algorithm

Theoretical Question 24

The training algorithm alternates between a Viterbi step and an OC-SVM step. What would happen if the algorithm were run for only one iteration ($M = 1$)? How does running more iterations potentially improve the learned model?

اگر الگوریتم فقط یک بار تکرار شود مدل فقط یکبار ترین میشود ان هم با پارامتر های اولیه HMM که خودمان به مدل دادیم. یعنی در ابتدا پارامتر های انتشار را با k -mean مقداردهی کردیم و ماتریس انتقال را نیز با فرض پایداری بیشتر سیستم که یعنی احتمال ماندن مدل در حالتی که الان هست بیشتر باشد مقداردهی شده و برای مثال احتمال شروع حالت های پنهان را نیز یکنواخت در نظر گرفتیم.

سپس در یک تکرار الگوریتم ویتربی برای دنیاله های آموزشی پیاده سازی میشود و مسیرهای پنهان فقط با استفاده از HMM اولیه که خودمان تخمین زدیم تخمین زده میشوند و پس از ان ویژگی مشترک هر دنیاله ساخته شده و سپس پارامتر های HMM با استفاده از مسیر هایی که در مرحله قبل تخمین زدیم بروزرسانی میشود و OC-SVM روی ویژگی های مشترک ترین میشود.

در نتیجه، اگر $(M=1)$ باشد، OC-SVM بر اساس مسیرهای پنهانی آموزش می بیند که مستقیماً از مقداردهی اولیهی HMM حاصل شده اند. اگر این مقداردهی اولیه دقیق نباشد، ممکن است بعضی از مشاهدات به حالت های اشتباه نسبت داده شوند. در این صورت، ویژگی های انتقال و انتشار نیز به درستی ساخته نمی شوند و مدل نهایی می تواند کیفیت پایینی داشته باشد.

حال تکرار این فرایند و افزایش تعداد اپدیت های پارامتر های HMM میتواند مفید باشد چون هر بار پارامتر های جدید از روی داده های آموزشی و مسیر های قبلی محاسبه میشود و احتمالاً دنیاله جدید تولید شده دقیق تر از دنیاله قبلی باشد چون اینگونه OC-SVM روی نمایش دقیق تری از دنیاله ها آموزش می بیند.

در واقع با هر بار تکرار الگوریتم ما هم مسیر های پنهان را بهتر میکنیم و ویتربی مسیر های پنهان سازگارتری برای داده های پیدا میکند و هم پارامتر های بهتری برای HMM پیدا میکند و تخمین بهتری برای پارامتر های ان بدست می آورد و در نهایت نیز باعث بهتر شدن ویژگی های ورودی OC-SVM میشود چون ویژگی های ورودی به مسیر های پنهان مربوط بودند و مسیر های دقیق تر باعث تولید بردار ویژگی بهتری میشود و پس OC-SVM بهتر و دقیق تر عمل میکند. چند تکرار می تواند وابستگی مدل به مقداردهی اولیه را کاهش دهد و تعریف سازگارتری از رفتار نرمال ایجاد کند.

Simulation Question 10

Implement the simplified HMAD training loop, Algorithm 1, in Python with $K = 2$ hidden states and Gaussian emissions.

- (a) Initialize emission parameters using K -means with $K = 2$, fitted on all individual time steps pooled from the training sequences.

Set the initial transition matrix such that

$$A_{kk} = 0.9$$

and use equal off-diagonal entries.

- (b) Run the training loop for up to $M = 10$ iterations.

After each iteration, record:

- (i) the fraction of hidden state assignments that changed from the previous iteration;
- (ii) the mean OC-SVM decision score on the training set.

- (c) Plot both quantities against the iteration number.

At what iteration does the hidden state assignment stabilize? Does the mean OC-SVM decision score become more stable across iterations?

- (d) Print the learned emission means μ_1, μ_2 and the transition matrix A at convergence. Do they match the structure of the data-generating process?

در این شبیه‌سازی، الگوریتم ساده‌شده *HMAD* برای یادگیری ساختار زمانی دنباله‌ها پیاده‌سازی شد. مدل مورد استفاده شامل دو حالت مخفی و انتشارهای گاوسی است.

a) در مرحله مقداردهی اولیه، پارامترهای انتشار مدل با استفاده از الگوریتم *K-means* با

$$K = 2$$

روی تمام نقاط زمانی داده‌های آموزش مقداردهی شدند. ماتریس انتقال اولیه برای هر دو دیتاست به صورت زیر در نظر گرفته شد:

$$A = \begin{bmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{bmatrix}$$

و احتمال اولیه حالت‌ها برابر بود با:

$$\pi = [0.5, 0.5]$$

پارامترهای اولیه انتشار برای دو دیتاست در جدول زیر آورده شده‌اند.

جدول ۱۲: *Initial Gaussian emission parameters*

Parameter	ECG5000	Daphnet FOG
μ_1	0.1753	0.0179
μ_2	-2.8316	23.3257
σ_1^2	0.4577	0.0826
σ_2^2	1.1243	18.4125

b) الگوریتم آموزش برای

$$M = 10$$

اجرا شد. در هر استپ دو مقدار ثابت گردید:

- تغییرات نسبت به مرحله قبل.
 - میانگین مقدار *OC-SVM decision score* روی داده‌های آموزشی.
- نتایج روند آموزش برای دیتاست *ECG5000* در جدول زیر آورده شده است.

جدول ۱۳: *HMAD training progress for ECG5000*

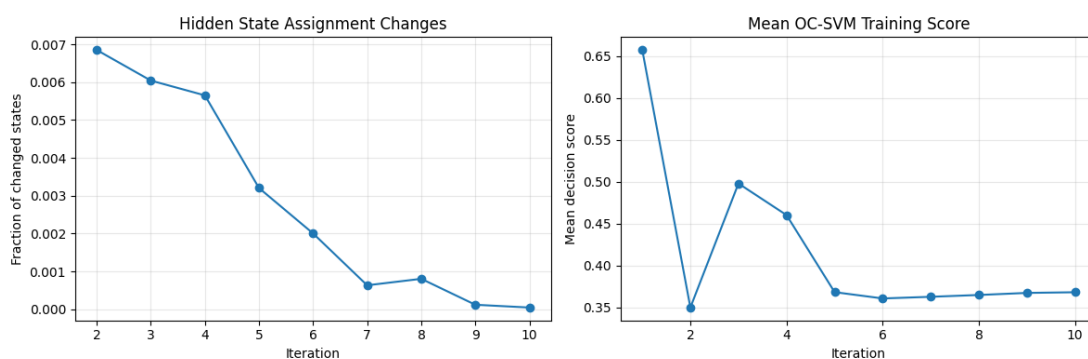
Iteration	Changed State Fraction	Mean OC-SVM Score
۱	-	0.657256
۲	0.006849	0.350100
۳	0.006042	0.498019
۴	0.005651	0.460353
۵	0.003205	0.368455
۶	0.002006	0.360932
۷	0.000636	0.362906
۸	0.000807	0.365079
۹	0.000122	0.367541
۱۰	0.000049	0.368330

نتایج آموزش برای دیتاست *Daphnet FOG* به صورت زیر است.

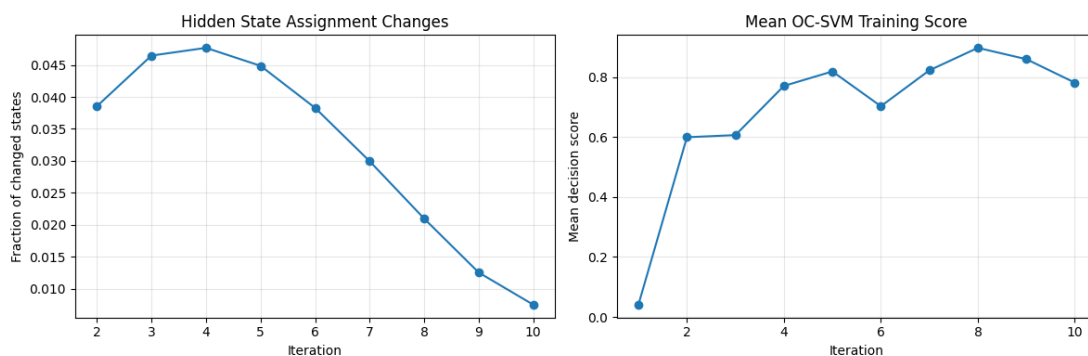
جدول ۱۴: *HMA training progress for Daphnet FOG*

Iteration	Changed State Fraction	Mean OC-SVM Score
۱	–	0.040861
۲	0.038542	0.599186
۳	0.046465	0.606256
۴	0.047676	0.770256
۵	0.044863	0.818504
۶	0.038281	0.702581
۷	0.029935	0.822213
۸	0.020951	0.897137
۹	0.012513	0.859426
۱۰	0.007480	0.781187

ج) برای بررسی روند همگرایی، تغییرات حالت‌های مخفی و میانگین *OC-SVM decision score* بر حسب شماره مرحله رسم شدند.



شکل ۱۱: *HMA convergence behavior for ECG5000*



شکل ۱۲: *HMA convergence behavior for Daphnet FOG*

در دیتاست *ECG5000* تغییرات به سرعت رخ داده و از مرحله ۷ به بعد تقریباً ثابت شده و پایدار شده در دیتاست *Daphnet FOG* با سرعت کمتری کاهش یافته و حتی تا مرحله ۱۰ هم کمی تغییرات داریم به پایداری کامل نرسیدیم میانگین *OC-SVM decision score* در هر دو دیتاست پس از مراحلی به پایداری رسیده

d) پس از پایان آموزش، پارامترهای نهایی مدل *Gaussian HMM* به صورت زیر به دست آمدند.

جدول ۱۵: *Learned emission means at convergence*

Parameter	ECG5000	Daphnet FOG
μ_1	0.2076	-0.0243
μ_2	-2.0925	0.1415

ماتریس انتقال نهایی برای دو دیتاست:

$$A_{\text{ECG5000}} = \begin{bmatrix} 0.9981 & 0.0019 \\ 0.0892 & 0.9108 \end{bmatrix}$$

$$A_{\text{Daphnet}} = \begin{bmatrix} 0.9814 & 0.0186 \\ 0.0459 & 0.9541 \end{bmatrix}$$

مقادیر نهایی ماتریس انتقال نشان می‌دهند که در هر دو دیتاست احتمال باقی ماندن در همان حالت بیشتر از احتمال تغییر حالت است. این موضوع با فرض وجود ساختار زمانی در داده‌ها سازگار است. همچنین میانگین‌های انتشار یادگرفته‌شده نشان می‌دهند که مدل توانسته است دو ناحیه متفاوت از فضای داده را به عنوان دو حالت مجزا شناسایی کند.

۴.۵ Prediction

Theoretical Question 25

During prediction, why is it necessary to run Viterbi decoding on the test sequence rather than using, for example, a fixed or randomly assigned hidden state sequence? What would go wrong if you skipped the Viterbi step and simply used

$$z = (1, 1, \dots, 1)$$

for every test sequence?

در مرحله پیش‌بینی، برای هر دنباله آزمایشی فقط مشاهدات (x) را داریم و دنباله حالت‌های پنهان (z) مشخص نیست. از طرفی بردار ویژگی مشترک $\Psi(x, z)$ هم به مشاهدات و هم به مسیر حالت‌های پنهان وابسته است. بنابراین ابتدا باید الگوریتم ویتربی را با پارامترهای HMM آموزش‌دیده اجرا کنیم تا محتمل‌ترین مسیر پنهان به دست آید:

$$z^* = \arg \max_z P(x, z \mid \pi, A, B).$$

سپس با استفاده از این مسیر، ویژگی‌های انتقال و انتشار دنباله محاسبه می‌شوند و OC-SVM امتیاز نهایی را به دست می‌آورد:

$$f(x) = \langle w, \Psi(x, z^*) \rangle - \rho.$$

اگر مرحله ویتربی را حذف کنیم و برای همه دنباله‌ها قرار دهیم

$$z = (1, 1, \dots, 1),$$

مدل به اشتباه فرض می‌کند سیستم در تمام زمان‌ها در حالت اول بوده است. در نتیجه، ویژگی‌های انتقال به صورت

$$c_{11} = T - 1, \quad c_{12} = c_{21} = c_{22} = 0$$

درمی‌آیند و تمام مشاهدات نیز به حالت اول نسبت داده می‌شوند؛ یعنی

$$e_1 = \sum_{t=1}^T \phi(x_t), \quad e_2 = 0.$$

بنابراین اطلاعات مربوط به تغییر حالت‌ها و نوع مشاهدات تولید شده در هر حالت از بین می‌رود. همچنین ویژگی‌های داده آزمایشی دیگر با ویژگی‌هایی که OC-SVM در زمان آموزش و با استفاده از مسیرهای ویتربی دیده است سازگار نخواهند بود. در نتیجه ممکن است یک دنباله نرمال به اشتباه ناهنجار تشخیص داده شود یا یک دنباله ناهنجار به دلیل حذف ساختار زمانی، نرمال به نظر برسد.

پس ویتربی لازم است تا برای هر دنباله آزمایشی یک مسیر پنهان متناسب با خود آن دنباله پیدا کند و بردار $\Psi(x, z^*)$ به درستی ساختار زمانی، انتقال‌ها و مشاهدات مربوط به هر حالت را نمایش دهد. استفاده از مسیر ثابت یا تصادفی باعث تولید ویژگی‌های اشتباه و امتیازهای ناهنجاری غیرقابل اعتماد می‌شود. پس نمی‌توان از یک دنباله‌ی حالت پنهان ثابت یا تصادفی استفاده کرد زیرا مشکلاتی مانند از بین رفتن اطلاعات زمانی که باعث می‌شود مدل دیگر تفاوت میان یک دنباله پایدار و یک دنباله با انتقال‌های غیرعادی را تشخیص ندهد. برای مثال:

فرض کنیم یک دنباله‌ی نرمال مسیر واقعی زیر را داشته باشد:

$$(1, 1, 1, 2, 2, 2).$$

در این مسیر، سیستم ابتدا در حالت اول و سپس در حالت دوم قرار دارد و یک انتقال طبیعی از حالت (۱) به حالت (۲) رخ می‌دهد. اما با انتخاب ثابت

$$z = (1, 1, 1, 1, 1, 1),$$

این انتقال کاملاً حذف می‌شود. در نتیجه، ویژگی c_{12} که باید برابر یک باشد، صفر در نظر گرفته می‌شود و تمام مشاهدات حالت دوم نیز به حالت اول نسبت داده می‌شوند.

و می‌تواند از طرفی باعث ناسازگاری با ویژگی‌های آموزشی می‌شود و حالات نسبت داده شده به مشاهدات اشتباه باشند زیرا اگر در مرحله آزمایش به جای ویتربی از مسیر ثابت استفاده کنیم نوع داده‌های آموزشی با نمایش حالت‌های پنهان آنها متفاوت خواهد بود و در نتیجه حتی ممکن است دنباله نرمال باشد اما باز هم امتیاز پایینی دریافت کند و ناهنجار گزارش شود و یا دنباله واقعاً ناهنجار به دلیل حذف اطلاعات انتقالی آن که قرار بود به وسیله دنباله حالات z بیان شود شبیه نرمال به نظر برسد. چون مدل آموزشی انتظار ندارد حالت اول مقادیر نزدیک پنج تولید کند. بنابراین ممکن است یک دنباله‌ی نرمال صرفاً به دلیل تخصیص اشتباه حالت‌ها، ناهنجار تشخیص داده شود.

از طرفی در نظر گرفتن دنباله‌های تصادفی نیز مناسب نیست زیرا برای یک دنباله از مشاهدات ثابت در اجراهای مختلف به دلیل در نظر گرفتن دنباله‌های رندوم متفاوت به نتیجه متفاوت می‌تواند برسد و در نتیجه امتیاز ناهنجاری برای این دنباله ناپایدار است و همان مشکل قبلی نیز کم‌کم وجود دارد که ممکن است دنباله مشاهدات نرمال باشد ولی اختصاص حالت به آنها که به صورت تصادفی انجام شده باعث شود ناهنجار تشخیص داده شود زیرا حالت مناسب به داده‌ها اختصاص داده نشده و مدل دیگر قابل اعتماد نخواهد بود.

Simulation Question 11

Using the trained HMAD model from the previous simulation question, implement the full prediction pipeline and evaluate it on the held-out test set.

- For each test sequence x , run Viterbi decoding to obtain z^* , compute $\Psi(x, z^*)$, and evaluate the OC-SVM score $f(x)$.
- Report ROC-AUC on the test set and compare it to the OC-SVM baseline from Section 3.

(c) To verify that Viterbi matters, repeat the evaluation using the trivial state assignment

$$z_t = 1$$

for all t and all sequences, thereby skipping the Viterbi step.

How does the AUC change? Does this confirm or contradict your answer to the theoretical question above?

(d) Plot the Viterbi-decoded hidden state path z^* for one correctly classified normal sequence and one correctly classified anomalous sequence side by side.

Does the decoded path visually align with the anomaly block?

در این شبیه‌سازی، مرحله پیش‌بینی کامل مدل $HMAD$ با استفاده از پارامترهای یادگرفته‌شده در سوال قبل روی مجموعه آزمون اجرا شد. هدف این بخش بررسی تأثیر استفاده از مسیر حالت‌های مخفی استخراج‌شده توسط $Viterbi$ $decoding$ در مقایسه با مدل پایه $OC-SVM$ است.

a) برای هر دنباله آزمون، ابتدا مسیر بهینه حالت‌های مخفی با استفاده از الگوریتم $Viterbi$ محاسبه شد سپس با استفاده از این مسیر، بردار ویژگی مشترک

$$\Psi(x, z^*)$$

محاسبه می‌شود. این بردار شامل ویژگی‌های حالت‌های مخفی است. در نهایت ویژگی‌های استخراج‌شده پس از استانداردسازی به مدل $OC-SVM$ داده شدند و امتیاز ناهنجاری هر دنباله محاسبه شد.

جدول ۱۶: $HMAD$ test feature extraction details

Dataset	Number of Test Sequences	Test Feature Shape
ECG5000	4500	4500×6
Daphnet FOG	600	600×6

b) عملکرد مدل $HMAD$ با مدل پایه $OC-SVM$ مقایسه شد. نتایج $ROC-AUC$ برای دو دیتاست در جدول زیر گزارش شده است.

جدول ۱۷: Comparison between $OC-SVM$ baseline and $HMAD$

Dataset	Method	ROC-AUC
ECG5000	$OC-SVM$ Baseline	0.984025
ECG5000	$HMAD + Viterbi$	0.933161
Daphnet FOG	$OC-SVM$ Baseline	0.375309
Daphnet FOG	$HMAD + Viterbi$	0.647469

نتایج نشان می‌دهد که عملکرد $HMAD$ به ساختار دیتاست وابسته است. در دیتاست $ECG5000$ مدل پایه $OC-SVM$ عملکرد بالاتری نسبت به $HMAD$ داشت، در حالی که در دیتاست $Daphnet FOG$ استفاده از ویژگی‌های ساختاری استخراج‌شده توسط $HMAD$ باعث افزایش قابل توجه $ROC-AUC$ شد.

c) برای بررسی اهمیت مرحله $Viterbi$ decoding یک آزمایش کنترلی انجام شد. در این حالت، مسیر حالت‌های مخفی ثابت در نظر گرفته شد:

$$z_t = 1, \quad \forall t$$

و بدون انجام مرحله *Viterbi* ویژگی‌های مشترک محاسبه شدند. نتایج مقایسه در جدول زیر آمده است.

جدول ۱۸: Effect of Viterbi decoding on HMAD performance

Dataset	Method	ROC-AUC
ECG5000	Without Viterbi	0.496717
ECG5000	With Viterbi	0.933161
Daphnet FOG	Without Viterbi	0.602222
Daphnet FOG	With Viterbi	0.647469

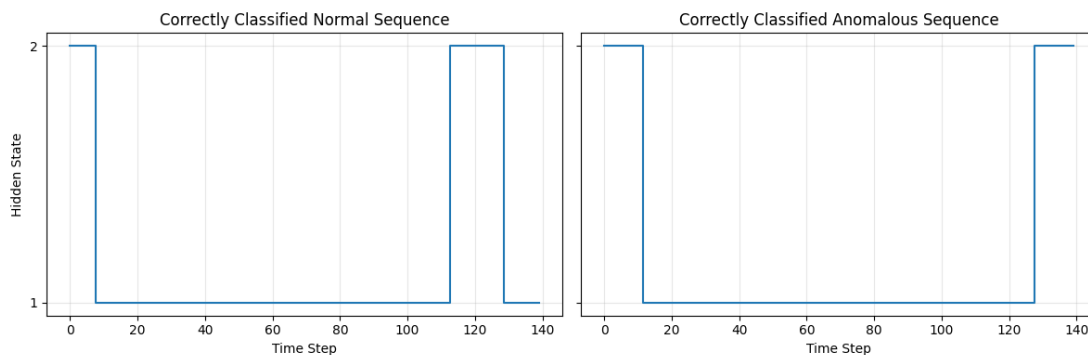
در هر دو دیتاست حذف مرحله *Viterbi* باعث کاهش شدید عملکرد شد:

0.933161	Remove Viterbi	0.496717
----------	----------------	----------

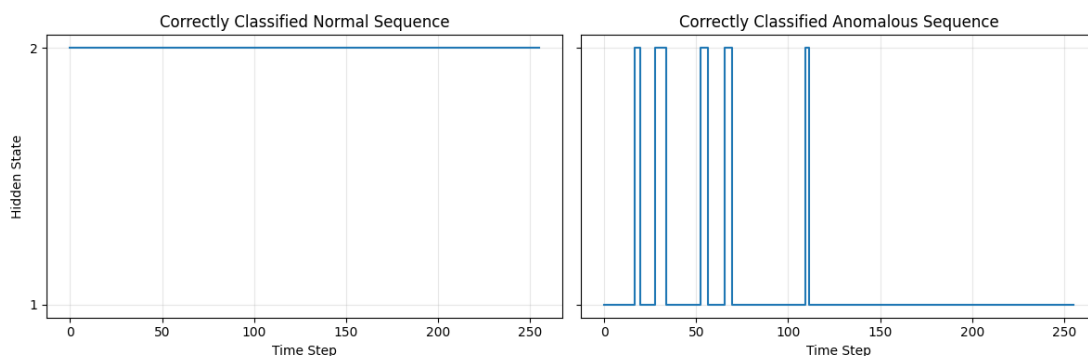
0.647469	Remove Viterbi	0.602222
----------	----------------	----------

این نتیجه نشان می‌دهد که تخصیص صحیح حالت‌های مخفی توسط *Viterbi* نقش مهمی در استخراج ویژگی‌های زمانی دنباله دارد.

d) برای بررسی رفتار مسیر حالت‌های مخفی، یک نمونه نرمال و یک نمونه ناهنجار که به درستی طبقه‌بندی شده‌اند انتخاب و مسیر حالت‌های استخراج‌شده توسط *Viterbi* رسم شد.



شکل ۱۳: Viterbi hidden state paths for ECG5000.



شکل ۱۴: Viterbi hidden state paths for Daphnet FOG.

در نمونه‌های ایراد دار مشاهده می‌شود که مسیر حالت‌های مخفی تغییراتی متناسب با بخش‌های غیرعادی دنباله نشان می‌دهد. این موضوع بیان می‌کند که مدل *HMAD* علاوه بر مقدار مشاهده‌ها، قادر است تغییرات ساختاری و زمانی داده را نیز در فرآیند تشخیص ناهنجاری لحاظ کند.

۶ Conclusion

در دیتاست *ECG5000* مدل *OC-SVM* بهترین عملکرد را ارائه داد. این نتیجه نشان می‌دهد که در این دیتاست، تفاوت میان دنباله‌های نرمال و ناهنجار تا حد زیادی در مقادیر مشاهده‌شده قابل تشخیص است و استفاده از ساختار زمانی اضافی در مدل *HMAD* بهبود بیشتری ایجاد نکرده است. در دیتاست *Daphnet FOG* استفاده از *HMAD* باعث افزایش عملکرد نسبت به *OC-SVM* شد. این نتیجه نشان می‌دهد که اطلاعات زمانی و الگوی انتقال بین حالت‌ها نقش مهمی در تشخیص ناهنجاری‌های این دیتاست دارند. مدل *Viterbi* توانست مسیر حالت مخفی مناسبی استخراج کند و ویژگی‌هایی را ایجاد کند که برای تشخیص ناهنجاری مفیدتر بودند.